IMPERIAL

IMPERIAL COLLEGE LONDON

DEPARTMENT OF MATHEMATICS

MSCI RESEARCH PROJECT

Variational and Neural Network Methods for American Put Option Pricing

Author:
James Wu

Supervisor(s):
Harry Zheng

Submitted in partial fulfillment of the requirements for the MSci in Mathematics at Imperial College London

March 18, 2026

Abstract

Type your abstract here. The abstract is a summary of the contents of the project. It should be brief but informative, and should avoid technicalities as far as possible.

Acknowledgments

Plagiarism statement

The work contained in this thesis is my own work unless otherwise stated.

Signature: James Wu
Date: March 18, 2026

Contents

Chapter 1

Introduction

The problem of correctly assigning the price of an option on a stock is a widely-researched field and is essential in minimising market inefficiencies. Black and Scholes have developed a closed-form solution to price the European put and call options [1], but their theory can be extended to American options as well. American options allow holders of the contract to exercise the option at any time before expiry, which turns our pricing problem into an optimal stopping problem.

In this study, we will specifically focus on put options on stocks paying no dividends, as it's well-known that it is always optimal to wait until expiry to exercise a call option provided there are no dividends [need to cite], and hence the American and European prices coincide. For an asset S, the put option has the payoff at expiry

$$g(S) = (K - S)^+$$

where K is the pre-determined strike price.

[Talk about the structure of the paper here (to be completed after more writeup is done)]

1.1 Probabilistic Setup

1.1.1 Risk-neutral measure and arbitrage-free pricing

The market in the real world is not arbitrage free, however, so there are implicit assumptions that have been made that we assume hold across the models that we'll explore:

- Shares of stock can be infinitely subdivided
- Interest rates for investing and borrowing are the same (in reality, the rate for borrowing is generally higher)
- The selling and buying prices coincide (i.e. there is no bid-ask spread)
- Buying and selling causes no friction in the market (no transaction costs, taxes, etc)
- A trader can long and short any amount of one stock, and borrow any amount from the bank

1.1.2 Black Scholes Model

As before, let $(\Omega, \mathcal{F}, \mathcal{F}_t)_{t\in[0,T]}, (W_t)_{t\in[0,T]}$ be a Wiener space as before where W_t is a Brownian motion, $\mathcal{F}_t$ is the natural filtration of W_t and $\mathcal{F} = \mathcal{F}_T$. Our market will contain two underlying securities:

- The risk-free asset, described by a deterministic function

$$\mathrm{d}B_t = rB_t\mathrm{d}t$$

  where for convenience, we set $B_0 = 1$ and r is the constant risk-free interest rate with continuous compounding. This has the solution $B_t = e^{rt}$ for $t \geq 0$.

- The risky asset S_t, which has the price process of a geometric Brownian motion (GBM), which satisfies the stochastic differential equation

$$\mathrm{d}S_t = \mu S_t\mathrm{d}t + \sigma S_t\mathrm{d}W_t \qquad 0 \leq t \leq T$$

  for a given drift μ and volatility σ. We can solve this SDE to yield the unique solution

$$S_t = S_0 \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)t + \sigma W_t\right) \qquad 0 \leq t \leq T$$

Consider the discounted stock price

$$\tilde{S}_t := \frac{S_t}{B_t} = e^{-rt}S_t$$

and apply Itô's lemma to get a new SDE in terms of the discounted stock price

$$\mathrm{d}\tilde{S}_t = \tilde{S}_t[(\mu - r)\mathrm{d}t + \sigma\mathrm{d}W_t^{\mathbb{P}}]$$

which means that under the physical measure $\mathbb{P}$, $\tilde{S}_t$ has nonzero drift unless $\mu = r$, and hence not a martingale. So to enforce no arbitrage in our pricing model, we need a new equivalent probability measure, name it $\mathbb{Q}$, under which $\tilde{S}_t$ is a martingale.

Let's define the quantity

$$c := \frac{\mu - r}{\sigma}$$

which measures how much excess drift per unit volatility the stock earns under the physical measure $\mathbb{P}$. Then we can define the Radon-Nikodym derivative

$$\left.\frac{\mathrm{d}\mathbb{Q}}{\mathrm{d}\mathbb{P}}\right|_{\mathcal{F}_t} = Z_t := \exp\left(-cW_t^{\mathbb{P}} - \frac{1}{2}c^2t\right)$$

which satisfies Novikov's condition as c is constant so $\mathbb{Q}$ is well defined. The new process

$$W_t^{\mathbb{Q}} := W_t^{\mathbb{P}} + ct$$

is a Brownian motion under $\mathbb{Q}$ by Girsanov's theorem. If we substitute this into the original SDE, we obtain

$$\begin{aligned}\frac{\mathrm{d}S_t}{S_t} &= \mu\mathrm{d}t + \sigma(W_t^{\mathbb{Q}} - c\mathrm{d}t)\\ &= (\mu - \sigma c)\mathrm{d}t + \sigma\mathrm{d}W_t^{\mathbb{Q}}\\ &= r\mathrm{d}t + \sigma\mathrm{d}W_t^{\mathbb{Q}}\end{aligned}$$

So now under $\mathbb{Q}$, we have $\mathrm{d}\tilde{S}_t = \sigma \tilde{S}_t \mathrm{d}W_t^{\mathbb{Q}}$ which has zero drift, and hence $\tilde{S}_t$ is a $\mathbb{Q}$-martingale.

Intuitively, this change of measure reweights possible stock paths so that there is no more risk premium, and all assets earn the risk-free rate in expectation after discounting.

The stopping problem we would like to solve can now be mathematically formalised as

$$\sup_{\tau \in [t,T]} \mathbb{E}[e^{-r(\tau - t)} g(S_\tau) \mid S_t = S]$$

where τ is a stopping time, and $g(\cdot)$ is the payoff function.

1.1.3 Product of multiple assets

As we would be pricing put options on multiple assets later, it would be useful to derive an equivalent 1-asset formulation that we could use to benchmark our results. As we model the stock price process to be a geometric Brownian motion, we can consider reformulating the product of multiple correlated assets.

To set up our model, let each asset S^i satisfy under the risk-free measure $\mathbb{Q}$, for $i = 1, \ldots, k$,

$$\mathrm{d}S_t^i = rS_t^i \mathrm{d}t + \sigma_i S_t^i \mathrm{d}W_t^i$$

with $\langle W^i, W^j \rangle_t := \mathbb{E}\big[\mathrm{d}W_t^i \mathrm{d}W_t^j\big] = \rho_{ij} \mathrm{d}t$ for all i, j pairs. The log representation is

$$\ln S_t^i = \ln S_0^t + \int_0^t \left(r - \frac{1}{2}\sigma_i^2 \right) \mathrm{d}s + \int_0^t \sigma_i \, \mathrm{d}W_s^i$$

Define our new asset X to be

$$X_t := \prod_{i=1}^k S_t^i$$

and hence we have

$$\ln X_t = \sum_{i=1}^k \ln S_t^i = \ln X_0 + \int_0^t \left(kr - \frac{1}{2} \sum_{i=1}^k \sigma_i^2 \right) \mathrm{d}s + \sum_{i=1}^k \int_0^t \sigma_i \, \mathrm{d}W_s^i$$

take the differential:

$$\mathrm{d} \ln X_t = \left(kr - \frac{1}{2} \sum_{i=1}^k \sigma_i^2 \right) \mathrm{d}t + \sum_{i=1}^k \sigma_i \, \mathrm{d}W_s^i$$

The quadratic variation of the martingale term is

$$\left\langle \sum_{i=1}^k \sigma_i W^i, \sum_{j=1}^k \sigma_j W^j \right\rangle_t = \sum_{i=1}^k \sum_{j=1}^k \sigma_i \sigma_j \rho_{ij} t =: \sigma_X^2 t$$

i.e. $\sigma_X^2 = \sum_{i=1}^k \sum_{j=1}^k \sigma_i \sigma_j \rho_{ij}$. So we can express the second term in terms of a new Brownian motion $\widetilde{W}$:

$$\sum_{i=1}^k \sigma_i \, \mathrm{d}W_s^i = \mathrm{d}\widetilde{W}_t$$

Going back, we have

$$\mathrm{d}\ln X_t = \left(kr - \frac{1}{2}\sum_{i=1}^{k}\sigma_i^2\right)\mathrm{d}t + \sigma_X \mathrm{d}\widetilde{W}_t$$

Exponentiating using Ito's formula gets us

$$\mathrm{d}X_t = X_t\left[\left(kr - \frac{1}{2}\sum_{i=1}^{k}\sigma_i^2\right)\mathrm{d}t + \sigma_X \mathrm{d}\widetilde{W}_t\right] + \frac{1}{2}X_t\sigma_X^2\mathrm{d}t$$

which we can rearrange to get

$$\mathrm{d}X_t = \left(kr - \frac{1}{2}\sum_{i=1}^{k}\sigma_i^2 + \frac{1}{2}\sigma_X^2\right)X_t\mathrm{d}t + \sigma_X X_t\,\mathrm{d}\widetilde{W}_t$$

which is a geometric Brownian motion with volatility σ_X and drift r_X where

$$\begin{aligned} r_X &= kr - \frac{1}{2}\sum_{i=1}^{k}\sigma_i^2 + \frac{1}{2}\sigma_X^2 \\ &= kr + \sum_{i<j}\sigma_i\sigma_j\rho_{ij} \end{aligned}$$

by matching coefficients. We can verify the equivalence by simulating many price paths for the assets and comparing their product against the simulations of price paths for the equivalent asset. Using the initial values

$$r = 0.05 \qquad S_0 = (10, 5, 3)^\top \qquad \sigma = (0.2, 0.3, 0.1)^\top \qquad \mathrm{R} = \begin{pmatrix} 1 & 0.6 & -0.2 \\ 0.6 & 1 & 0.4 \\ -0.2 & 0.4 & 1 \end{pmatrix}$$

where R is the correlation matrix, we can plot the histogram of final prices in Figure [1.1].

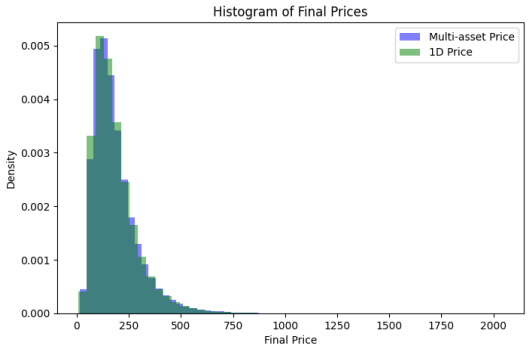

Figure 1.1 Histogram of final prices for product of assets and equivalent asset

They look very similar, and we can further statistically test if they come from the same distribution. Using Kolmogorov-Smirnov's 2-sample test on the null hypothesis that they come from the same distribution, we obtain a p-value of 0.8491, which does not provide sufficient evidence to reject the null hypothesis, which gives us stronger evidence that our derivation is correct.

Therefore, we will be able to price the put option on the product of multiple assets and compare that to the price of the put option on this single constructed asset as a benchmark.

Chapter 2

Numerical Methods

2.1 Tree Methods

2.2 Physics-Informed Neural Networks

2.2.1 Motivation

Our option pricing problem can alternatively be formulated as a variational inequality which takes the form of a partial differential equation that is independent from the payoff $g(\cdot)$. These PDEs can be of high dimension, and are thus difficult to solve using traditional numerical methods due to the curse of dimensionality. Traditional methods such as finite difference methods require discretisation of the domain and the number of grid points (and hence algorithm complexity) scales exponentially $\mathcal{O}(N^d)$ with the dimension d of the problem.

We can attempt a neural network solution to the PDEs, as neural networks are known to be universal approximators. That is, suppose we have the class of functions $\mathcal{F}$ over a compact set S that can be represented by a neural network with a given architecture, with a continuous target function $f^\star$ that we would like to approximate. Then for any target accuracy $\epsilon > 0$, there exists a function $f \in \mathcal{F}$ such that

$$\sup_{x \in S} |f(x) - f^\star(x)| \leq \epsilon \tag{2.1}$$

If we have a squashing activation function $\Psi(\cdot)$ that is non-decreasing and satisfies the limits

$$\Psi(-\infty) = 0, \quad \Psi(\infty) = 1 \tag{2.2}$$

then the class of functions $\mathcal{F}$ that can be represented by a neural network with one hidden layer and activation function Ψ is dense in the space of continuous functions on S and thus is a universal approximator [2].

Since the option price function is continuous in the time-asset price domain, we can use a neural network to find an approximation. This is favourable because the time complexity scales with the number of parameters, which is not exponential. To do this, we will use the framework of physics-informed neural networks (PINNs) to solve the PDEs, which have been shown to be effective for solving high-dimensional PDEs [3], by avoiding mesh construction and leveraging the interpolation capability of neural networks [4]. The loss function of the neural network is designed to penalise deviations from the PDE as well as deviations from the boundary conditions. By training the neural network to minimise this loss function, we can obtain an approximation

to the solution of the PDE. This method has been applied to option pricing problems in the literature, such as in [5] for the Black-Scholes model, but we will look to apply it to more complex models such as the Heston model as well as to options on multiple assets.

2.2.2 Methodology

We first provide a general setup for PINNs, before modifying it to tailor our specific problem of option pricing. Formally, suppose we have an unknown latent solution $u(t,x)$ which is governed by a PDE of the form

$$\frac{\partial u}{\partial t} + \mathcal{L}[u] = 0, \qquad t \in [0,T], x \in \Omega \tag{2.3}$$

where $\mathcal{L}$ is a (not necessarily linear) differential operator. Suppose additionally that this PDE is subject to the boundary conditions

$$u(0,x) = g(x), \qquad x \in \Omega \tag{2.4}$$

$$\mathcal{B}[u] = 0, \qquad t \in [0,T], x \in \partial\Omega \tag{2.5}$$

where $\mathcal{B}$ is a boundary operator. Then the unknown solution u can be approximated by a deep neural network $f_\theta(t,x)$ where θ are the parameters of the neural network, trained by minimising a loss function composed of deviations from the PDE residual, boundary conditions and initial conditions [6]:

$$L(\theta) = \lambda_r L_r(\theta) + \lambda_{bc} L_{bc}(\theta) + \lambda_{ic} L_{ic}(\theta) \tag{2.6}$$

where the loss components are mathematically defined as

$$L_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \mathcal{R}_\theta(t_r^i, x_r^i) \right|^2 \tag{2.7}$$

$$L_{bc}(\theta) = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} \left| \mathcal{B}[f_\theta](t_{bc}^i, x_{bc}^i) \right|^2 \tag{2.8}$$

$$L_{ic}(\theta) = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left| f_\theta(0, x_{ic}^i) - g(x_{ic}^i) \right|^2 \tag{2.9}$$

where the PDE residual $\mathcal{R}_\theta$ is defined as

$$\mathcal{R}_\theta(t,x) = \frac{\partial f_\theta}{\partial t}(t,x) + \mathcal{L}[f_\theta](t,x) \tag{2.10}$$

We additionally use the sampled points $\{x_{ic}^i\}_{i=1}^{N_{ic}}$ from the initial condition domain, $\{(t_{bc}^i, x_{bc}^i)\}_{i=1}^{N_{bc}}$ from the boundary condition domain and $\{(t_r^i, x_r^i)\}_{i=1}^{N_r}$ from the PDE domain, sampled uniformly in their respective domains at each iteration of the gradient descent algorithm.

There are different methods to choose the loss weights $\lambda_r, \lambda_{bc}, \lambda_{ic}$, such as using fixed weights or using adaptive weights that change during training. For simplicity, we will use fixed weights in the project.

For our specific problem of option pricing, we can modify the above setup to fit our variational inequality formulation. The PDE to solve is dependent on the model for the asset dynamics, and the boundary conditions are given by the payoff function of the option. In general, we will have to solve a problem of the form

$$\min\left\{ -\frac{\partial u}{\partial t} + \mathcal{L}[u], u - g \right\} = 0, \qquad t \in [0,T], x \in \Omega \tag{2.11}$$

$$u(T,x) = g(x), \qquad x \in \Omega \tag{2.12}$$

$$\mathcal{B}_k[u] = 0 \qquad t \in [0,T], x \in \partial\Omega, k = 1, \dots, K \tag{2.13}$$

where k indexes the K different boundary conditions. This is very similar to the general setup for PINNs, except that we have a variational inequality instead of a PDE, and we have a terminal condition instead of an initial condition. However, we can still use the framework by noting we can make the substitution $t \mapsto T - t$ to un-negate the negative partial derivative in the PDE term and to convert the terminal condition to an initial condition. The variational inequality can be handled by modifying the PDE residual loss (2.10) to

$$\mathcal{R}_\theta(t,x) = \min\left\{\frac{\partial f_\theta}{\partial t}(t,x) + \mathcal{L}[f_\theta](t,x), f_\theta(t,x) - g(x)\right\} \tag{2.14}$$

2.3 Sobolev Deep Neural Networks

Chapter 3

Binomial Tree Methods

3.1 Background

As the solutions to the optimal stopping problem do not have a closed form, we would require a benchmark for our numerical results for the different methods we are trying later in the study. We will utilise binomial trees for this for their simplicity and efficiency (it has time complexity $\mathcal{O}(n^2)$). It works by simulating all possible price paths given some modelling simplifications, and computing the price of the derivative using the martingale property with backwards induction.

To price an American put option with a binomial tree, we will use the binomial discrete-time model. This is an abstraction of the market dynamics, where the stock price can only move to one of two possible new prices in a time step.

To formalise this, let's fix a time horizon $T > 0$. For some depth $n \geq 1$, divide the interval $[0, T]$ into n even steps each of length $\Delta t = T/n$. The financial market will consist of

- A risk-free asset B_t which satisfies

  $$B_{i+1} = B_i e^{r\Delta t} \qquad B_0 = 1$$

  where r is the continuously-compounded risk-free rate, typically considered interest.

- A risky asset S_t which follows a multiplicative binomial evolution:

  $$S_{i+1} = \begin{cases} S_i u & \text{with probability } p \\ S_i d & \text{with probability } 1 - p \end{cases} \qquad S_0 > 0$$

  Here u and d are the model parameters determining the up and down factors of the stock price change per step, where we choose $u > 1$ and $d \in (0, 1)$. For an arbitrage free market, we should additionally ensure that

  $$u > e^{r\Delta t} > d$$

  for sufficiently small Δt.

For the absence of arbitrage, we require the discounted asset price to have zero drift (admitting a martingale measure). More precisely, let the discounted price be

$$\tilde{S}_i = e^{-ri\Delta t} S_i$$

We require some probability measure $\mathbb{Q}$, equivalent to the physical measure $\mathbb{P}$ (have the same non-zero support) such that

$$\mathbb{E}_{\mathbb{Q}}[\tilde{S}_{i+1}|\mathcal{F}_i] = \tilde{S}_i$$

To calculate the risk-neutral probability, we can apply this to our model:

$$e^{-r(i+1)\Delta t}\left[pS_iu + (1-p)S_id\right] = e^{-ri\Delta t}S_i$$

Collect the p terms and cancel out the discount factor to get

$$p(S_iu - S_id) + S_id = e^{r\Delta t}S_i$$

Divide both sides by S_i and rearrange to make p the subject to obtain

$$p = \frac{e^{r\Delta t} - d}{u - d}$$

which we will use in our binomial model.

We will adopt a backward induction approach to price the American put at time 0, since we already know the price at maturity, which is simply the payoff $g : \mathbb{R}^+ \to \mathbb{R}$ of the put option:

$$g(S_T) = (K - S_T)^+$$

As the discounted price process is a martingale under the risk-free measure $\mathbb{Q}$, the arbitrage-free value V_i of the derivative at time $i\Delta t$ is necessarily

$$V_i = \underbrace{e^{-r(T-i\Delta t)}}_{\text{discount}}\,\mathbb{E}_{\mathbb{Q}}[g(S_T)\,|\,\mathcal{F}_i]$$

3.2 Implementation

We will utilise the Cox-Ross-Rubinstein (CRR) Model [7] for our implementation, which chooses the parameters

$$u = e^{\sigma\sqrt{\Delta t}} \qquad d = e^{-\sigma\sqrt{\Delta t}}$$

satisfying the arbitrage free condition, and normalising the local variance of returns to the volatility parameter σ^2. They also conveniently multiply to 1.

To implement this, we need to first compute the price tree. This would be, for the price at step i after j up-moves (so clearly $i \geq j$)

$$S_{i,j} = S_0u^jd^{i-j}$$

As the payoff is known at maturity, we can simply set

$$V_n(j) = g(S_{n,j}) \qquad 0 \leq j \leq n$$

and recursively set

$$V_i(j) = e^{-r\Delta t}\big(p^* V_{i+1}(j+1) + (1-p^*)V_{i+1}(j)\big)$$

for $0 \leq j \leq i$. This would be it for European options, but since early exercise is allowed, we will compare this value with the payoff of the price at step i with j up-moves, which is just $f(S_{i,j})$. Thus the value is given by

$$V_i(j) = \max\left(e^{-r\Delta t}\big(p^* V_{i+1}(j+1) + (1-p^*)V_{i+1}(j)\big),\ g(S_{i,j})\right)$$

The root value $V_0(0)$ would be the arbitrage-free option price.

3.3 Visualisation

We are now ready to visualise our binomial tree model. Figure 3.1 plots the option values across different asset prices and different times. We can see that at $t = 1$, we are at expiry and hence the value is just the intrinsic value. However, the further we are from expiry, the more the option is worth. This is because the option has time value: the more time we have left to expiry, the more optionality we have and the more opportunity the asset has to be in the money. This additional optionality costs, and hence we can see that the put option loses value over time even if the asset price stays the same.

This phenomenon can be examined more closely in 3.2. We examine the structures for in the money, at the money and out of the money put options. We supply the European price for reference, and we can see that the European price indeed bounds the American price from below as we'd expect. We can see that the option loses value as we approach maturity. An interesting figure to examine is the in the money figure: we can see that the value flattens at 0.1, the intrinsic value of the option, at some time $t^\star \in (0, 1)$. This is the point of time beyond which it is always beneficial to perform early exercise, and hence the value of the option is just the intrinsic value. At times $0 < t < t^*$ the option is worth more as we have the option to wait, giving the option time value.

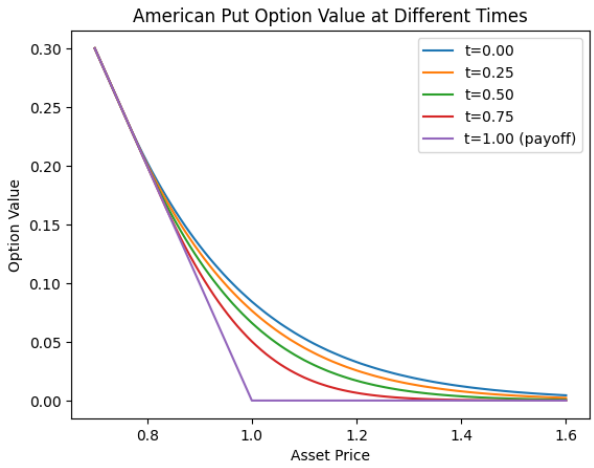

Figure 3.1 Option value on different asset prices across time

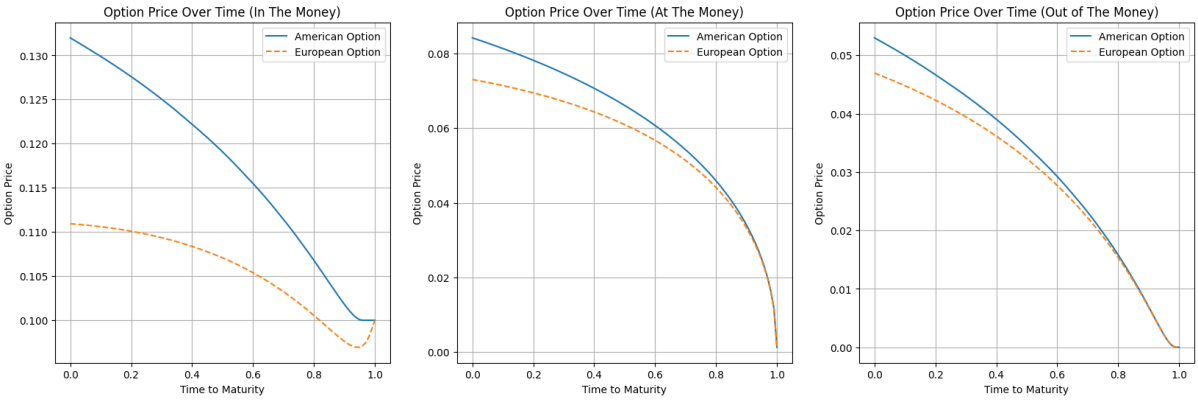

Figure 3.2 Decay in time value

Chapter 4

Neural Network Solution

We will now explore how to use deep neural networks to price our American put options. Using the Black-Scholes dynamics we can derive the variational equation, a partial differential equation that our solution must satisfy. As the solution of this PDE does not have a closed form in general, we aim to find an approximation to the solution by using a neural network. This can be done by customising a loss function as the variational equation, along with some boundary conditions that we know our value function must satisfy, with the hope that using optimisation techniques such as gradient descent, we can approach the solution numerically by descending the loss landscape.

4.1 Variational Equation

We will derive the variational equation in one dimension for simplicity, but the same steps can be followed for a multi-dimensional derivation, which would be covered when we determine the Black-Scholes operator.

Recall the value function of an American put with payoff g at time t with asset price $S_t = S$ is given by

$$v(t, S) = \sup_{\tau \in [t,T]} \mathbb{E}_t \big[e^{-r(\tau - t)} g(S_\tau) \big]$$

where τ is any stopping time in $[t, T]$ and $\mathbb{E}_t$ is conditional expectation at time t, given $S_t = S$. For an American option, we always have two options:

- Immediately exercise the option and get payoff $g(S)$. Since this is not necessarily optimal, the payoff must bound the value option from below:

  $$v(t, S) \geq g(S)$$

- Do nothing and let the option continue to time $t + h$ with $h > 0$ a very small number. With some derivation [8] we arrive at

  $$e^{rh} v(t, S) \geq \mathbb{E}_t \big[v(t + h, S_{t+h}) \big]$$

  and the inequality turns into an equality at the limit $\delta \to 0$.

We will additionally assume that the value function $v \in C^{1,2}$ (continuously differentiable with respect to t and twice continuously differentiable with respect to S) so that we can apply Itô's formula. Let's model the asset price S as a geometric Brownian motion satisfying

$$\mathrm{d}S_t = rS_t\mathrm{d}t + \sigma S_t \mathrm{d}W_t$$

Then by Ito's lemma and taking expectations of both sides we have

$$\mathbb{E}\big[v(t+\delta, S_{t+h})\big] = v(t,S) + \mathbb{E}_t\left[\int_t^{t+h}\left(\frac{\partial v}{\partial t}(u,S_u) + \frac{\partial v}{\partial S}(u,S_u)rS_u + \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(u,S_u)\sigma^2 S_u^2\right)\mathrm{d}u\right]$$

Now let's substitute into the LHS. By using a Taylor approximation $e^{rh} = 1+rh+o(h)$, cancelling $v(t,S)$ on both sides and dividing both sides by h we get

$$rv(t,S) + \frac{o(h)}{h} \geq \mathbb{E}_t\left[\frac{1}{h}\int_t^{t+h}\left(\frac{\partial v}{\partial t}(u,S_u) + \frac{\partial v}{\partial S}(u,S_u)rS_u + \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(u,S_u)\sigma^2 S_u^2\right)\mathrm{d}u\right]$$

Sending $h \to 0$ makes the higher order term $o(h)/h$ vanish, and the expectation of the integral to collapse to the integrand. Thus we obtain

$$rv(t,S) \geq \frac{\partial v}{\partial t}(t,S) + \frac{\partial v}{\partial S}(t,S)rS + \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(t,S)\sigma^2 S^2$$

or by defining the linear parabolic operator $\mathcal{G}$ [9]:

$$\mathcal{G}[v] = rv(t,S) - \frac{\partial v}{\partial t}(t,S) - \frac{\partial v}{\partial S}(t,S)rS - \frac{1}{2}\frac{\partial^2 v}{\partial S^2}(t,S)\sigma^2 S^2 \geq 0$$

which we could also write in terms of the Greeks:

$$\mathcal{G}[v] = rv(t,S) - \Theta - rS\Delta - \frac{1}{2}\sigma^2 S^2 \Gamma \geq 0$$

We can concisely express this inequality together with $(\dagger)$:

$$\min\left(\mathcal{G}[v], v(t,S) - g(S)\right) \geq 0$$

However, note that at time t, we must either exercise the option or do nothing and continue. One of these two options must be optimal, implying that one of the arguments must be 0. Thus we actually have an equality:

$$\min\left(\mathcal{G}[v], v(t,S) - g(S)\right) = 0$$

which is the *variational equation* [10] for the American option. Clearly, we have the terminal condition $v(T,S) = g(S)$.

4.2 Black-Scholes Operator

To obtain the variational equation for multiple assets, we can follow the same steps as we did for the one-dimensional case. Consider a derivative on k underlying assets whose price vector $S = (S^1, \dots, S^k)^\top$ follows a geometric Itô diffusion under the risk-neutral measure $\mathbb{Q}$. Formally, the asset prices solve the SDEs under $\mathbb{Q}$:

$$\mathrm{d}S_t^i = rS_t^i \mathrm{d}t + \sigma_i S_t^i \mathrm{d}W_t^i$$

where we can introduce correlation into the assets by

$$\mathbb{E}\big[\mathrm{d}W_t^i \mathrm{d}W_t^j\big] = \rho_{ij}\mathrm{d}t$$

As before, denote $g : \mathbb{R}_+^k \to \mathbb{R}$ the payoff on exercise and $T > 0$ the time of maturity. The value function stays

$$v(t, S) = \sup_{\tau \in [t,T]} \mathbb{E}_t\big[e^{-r(\tau - t)} g(S_\tau)\big]$$

By following the 1D case, should we choose not to exercise the option and proceed to $t + h$ for a small $h > 0$, we can apply Ito's lemma on multiple dimensions to yield

$$v(t + h, S_{t+h}) = v(t, S) + \int_t^{t+h} \left(\frac{\partial v}{\partial t}(u, S_u) + \mathcal{L}_u v(S_u) \right) \mathrm{d}u$$

where, for convenience, we have the Black-Scholes ordinary differential operator $\mathcal{L}$ acting on S with

$$\mathcal{L}_u v(S_u) = \sum_{i=1}^k \frac{\partial v}{\partial S^i}(u, S_u) r S_u + \frac{1}{2} \sum_{i=1}^k \sum_{j=1}^k \frac{\partial^2 v}{\partial S^i \partial S^j}(u, S_u) \sigma_i \sigma_j \rho_{ij} S_u^i S_u^j$$

and the linear parabolic operator $\mathcal{G}$ has the analogous definition as to the 1D case:

$$\mathcal{G}[v] := rv(t, S) - \frac{\partial v}{\partial t}(t, S) - \mathcal{L}_t v(S)$$

We can thus, using the same steps as the 1D case, yield the variational equation for multiple dimensions to be the same thing:

$$\min\left(\mathcal{G}[v], v(t, S) - g(S)\right) = 0$$

with the terminal condition $v(T, S) = g(S)$.

4.3 Implementation

To use a neural network to approximate the function $v(t, S)$, we need to establish a loss function that would help our estimate converge to the solution. As mentioned before, this loss function should take both the variational equation and the boundary conditions into account. The boundary conditions would of course depend on the payoff of the derivative specifically, but for an American put option it has the general structure:

- Variational loss (J_1): the value we obtain when we plug the current function represented by the neural network into the variational equation.

- Exercise loss (J_2): the value at the expiry should just be the payoff.

- Max boundary loss (J_3): the value of the derivative at the upper bound of the stock price should be 0 at any time, as we assume the option will expire out of the money.

- Min boundary loss (J_4): the value of the derivative at the lower bound of the stock price should be the payoff at any time, as early exercise is always good here.

Taking this into account, we will train our network over a set number of iterations where in each iteration, we will implement stochastic gradient descent by sampling a batch of B points for both the time and the stock prices, which would be used to approximate the loss that we descend, which is the weighted sum of the four components

$$J = \sum_{i=1}^{4} \lambda_i J_i$$

for $\boldsymbol{\lambda} = (\lambda_1, \ldots, \lambda_4) \in \Delta^4$, the unit simplex. These weights are to be tuned so that no component of the loss function is dominating the loss and we get a better fit.

4.3.1 One dimension

We will approximate the four loss components by sampling B points each epoch and computing the mean squared error of the loss contributions. For this, we will choose our batch size to be $B = 1000$ to obtain our sample points for epoch i

$$(t_1^{(i)}, \ldots, t_B^{(i)}) \in [0, T]^B$$
$$(S_1^{(i)}, \ldots, S_B^{(i)}) \in [S_{\text{min}}, S_{\text{max}}]^B$$

where $S_{\text{min}}, S_{\text{max}} \in [0, \infty)$ are parameters which we assume to be the minimum and maximum possible values of the stock respectively between now and expiry. To obtain these samples, we can consider various sampling schemes [add details]:

- Uniform distribution

- Truncated normal

With our batch of samples $(t_1^{(i)}, \ldots, t_B^{(i)})$ and $(S_1^{(i)}, \ldots, S_B^{(i)})$, the loss components in epoch i can be computed as the MSE:

$$J_1^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\left\|\min\left(\mathcal{G}[f], f(t_j^{(i)}, S_j^{(i)}) - g(S_j^{(i)})\right)\right\|_2^2$$

$$J_2^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\|f(T, S_j^{(i)}) - (K - S_j^{(i)})^+\|_2^2$$

$$J_3^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\|f(t_j^{(i)}, S_{\text{max}})\|_2^2$$

$$J_4^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\|f(t_j^{(i)}, S_{\text{min}}) - (K - S_{\text{min}})\|_2^2$$

with $J^{(i)} = \sum_{i=1}^{4}\lambda_i^{(i)}J_i^{(i)}$ as before, where our weights $\boldsymbol{\lambda}^{(i)} \in \Delta_4$ can either be constant or evolve dynamically. For simplicity, we choose them to be constant after tuning. We will use the ReLU activation function, the Adam optimiser with default parameters and learning rate 0.001.

4.3.2 Loss in multiple dimensions

Again, we will sample B points per epoch to approximate the loss components for each epoch. A batch size of $B = 1000$ works similarly well for our purposes. We will introduce the notation for our samples on k assets:

$$(t_1, \ldots, t_B) \in [0, T]^B$$
$$(\boldsymbol{S}_1, \ldots, \boldsymbol{S}_B) \in [S_{\text{min}}^1, S_{\text{max}}^1] \times \cdots \times [S_{\text{min}}^k, S_{\text{max}}^k]$$

where $\boldsymbol{S}_j = (S_j^1, \ldots, S_j^k)$. We would like to price the price of the put on the product of these k assets, so let's additionally introduce the product

$$X_j = \prod_{i=1}^{k} S_j^i$$

where we can additionally have $X_{\text{min}} = \prod_{i=1}^{k} S_{\text{min}}^i$ and $X_{\text{max}} = \prod_{i=1}^{k} S_{\text{max}}^i$. For simplicity, all of these samples will be generated iid and uniformly in their respected domains. Given these samples, the loss components can be extended to k dimensions:

$$J_1 = \frac{1}{B}\sum_{j=1}^{B}\|\min\left(\mathcal{G}[f], f(t_j, \boldsymbol{S}_j) - g(\boldsymbol{S}_j)\right\|_2^2$$

$$J_2 = \frac{1}{B}\sum_{j=1}^{B}\left\|f(T, \boldsymbol{S}_j) - (K - X_j)^+\right\|_2^2$$

$$J_3 = \frac{1}{B}\sum_{j=1}^{B}\left\|f(t_j, \boldsymbol{S}_j)\mathbb{I}_{\{X_j > 0.8X_{\text{max}}\}}\right\|_2^2$$

$$J_4 = \frac{1}{B}\sum_{j=1}^{B}\left\|\left(f(t_j, \boldsymbol{S}_j) - (K - X_j)^+\right)\mathbb{I}_{\{\exists i : S_j^i = S_{\text{min}}^i\}}\right\|_2^2$$

where $\mathbb{I}_A$ is the indicator function for the event A. The first two components, J_1 and J_2 are analogous to the 1D case. For the third component, we will only penalise if the product of the assets is close to some threshold, which we choose to be 0.8 of the maximum possible product of asset prices. The fourth component enforces immediate exercise as long as one of the assets is at its minimum theoretical price, which would be done by selecting an asset uniformly randomly in each sample and setting that to the corresponding modelled minimum price.

Again, our total loss for the epoch is the weighted sum of these four components, and we will again use the ReLU activation function, the Adam optimiser with default parameters and learning rate 0.001.

4.4 Numerical Results

4.5 Free Boundary

We are now interested in finding the free boundary of the American put option, which is the boundary separating the exercise region and the continuation region. For an initial visualisation, assume the risk-neutral dynamics of our asset S is modelled as a geometric brownian motion with constant risk-free interest rate r and volatility σ. As before, the value of the American put with strike K and maturity T is given by the optimal stopping problem

$$v(0,S) = \sup_{\tau \in [0,T]} \mathbb{E}\left[e^{-r\tau}(K - S_\tau)^+\right]$$

which is achieved by an optimal stopping time $\tau^\star$ taking the form

$$\tau^\star = \inf\{t \geq 0 : S(t) \leq b^\star(t)\}$$

for some optimal exercise boundary $b^\star(t)$ [11]. This is illustrated in Figure 4.1, where we simulate one realisation of the asset price and plot it against the exercise boundary, which is found using binary search on the binomial tree solution, where we have

$$b^\star(t) = \inf\{S : v(t,S) > g(S)\}$$

because as soon as the option has no time value, it is optimal to exercise.

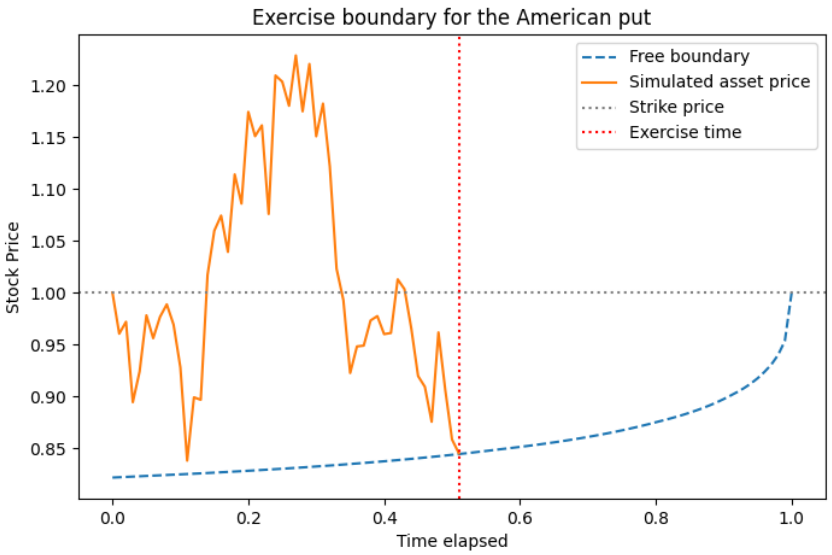

Figure 4.1 Exercise boundary for the American put

This formulation describes a hard classification problem, where we classify points in the time-stock price space binarily to be either in the continuation region or the exercise region. However, our numerical solutions to the pricing problem are not exact and there is some level of error in the solution that comes from the Monte Carlo simulations upon the space, the numerical differentiation used in training and the early stopping in the training process. Therefore, it would be more intuitive to perform a soft classification instead, where we assign probabilities to points in the time-stock price space to be in the continuation region or the exercise region, which would give us a more intuitive visualisation of the free boundary and how it evolves over

time. We will do this by extending hard classification: that is to say, the continuation region A is

$$A = \{(t, S) : u(t, S) > g(S)\} \cup \{(t, S) : g(S) = 0\}$$

where intuitively the first term indicates that we still have time value, and the second term immediately assigns at-the-money and out-of-the-money options in the continuation region as they are worthless under immediate exercise.

To extend the definition of the continuation region to a soft classification, we consider the sigmoid function $s : \mathbb{R} \to (0, 1)$ defined as

$$s(x) = \frac{1}{1 + e^{-x}}$$

and introducing two softness parameters $\tau_1, \tau_2 > 0$ to yield our continuation probability function on the relative difference from the payoff as

$$P_{\text{cont}}(d) = \begin{cases} s\left(\frac{d}{\tau_1}\right) & d \leq 0 \\ s\left(\frac{d}{\tau_2}\right) & d > 0 \end{cases}, \qquad d := \frac{u(t, S) - g(S)}{g(S)}$$

We introduce different softness paramaters depending on the sign of d because we would like different leniencies for when the value function is above or below the payoff. Value functions below the payoff arise under errors in the neural network solution and thus our continuation probability should be steeper.

These softness parameters can be determined by defining thresholds $\epsilon_1, \epsilon_2 > 0$ such that for $I \in (0.5, 1)$ we have

$$P_{\text{cont}}(-\epsilon_1) = 1 - I, \qquad P_{\text{cont}}(\epsilon_2) = I$$

which we could solve with some simple algebraic manipulation to get

$$\tau_1 = \frac{\epsilon_1}{\ln\left(\frac{I}{1-I}\right)}, \qquad \tau_2 = \frac{\epsilon_2}{\ln\left(\frac{I}{1-I}\right)}$$

These classifications are illustrated in Figure 4.2, where we can see that the continuation probabilities are close to 1 in the continuation region and close to 0 in the exercise region, with a smooth transition between the two regions. We can see that even though the neural network solution correctly captures the regional behaviour of the value function, it struggles with estimating the time value of the option, which can be observed by the fact that the free boundary curve is concave rather than convex.

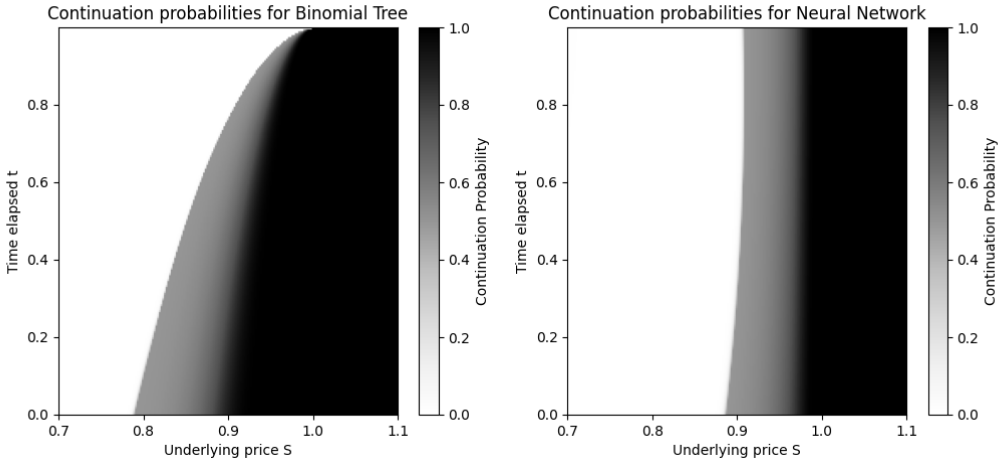

Figure 4.2 Continuation probabilities for the American put

Chapter 5

Fractional Sobolev Norm

We will now explore a different method to price American puts with neural networks, and this time we will work with Fractional Sobolev Norms to construct our loss function. The benefit of this is that we would be able to guarantee that our neural network converges to the true solution provided that its loss converges, and that our loss conditions apply for all situations and is not dependent on the payoff.

5.1 Preliminaries

5.1.1 Weak derivatives

Suppose $u \in L^1_{\text{loc}}(\Omega)$, a locally integrable function on Ω. This means that $u \in L^1(U)$ for every open set U such that the closure $\overline{U} \subset \Omega$ and $\overline{U}$ is a compact (closed and bounded) subset of $\mathbb{R}^n$.

[finish later]

5.1.2 Sobolev spaces

Here, we introduce Sobolev spaces of integer order, defined over an arbitrary domain $\Omega \subset \mathbb{R}^n$. These are vector subspaces of various Lebesgue spaces $L^p(\Omega)$, which is the class of all measurable functions u for which

$$\int_\Omega |u(x)|^p \, \mathrm{d}x < \infty$$

We first define the Sobolev norms, which is a functional $\|\cdot\|_{m,p}$ where $m \in \mathbb{N}_+$ and $1 \leq p \leq \infty$, as follows:

$$\|u\|_{m,p} = \left(\sum_{0 \leq |\alpha| \leq m} \|D^\alpha u\|_p^p \right)^{1/p} \qquad 1 \leq p < \infty$$
$$\|u\|_{m,\infty} = \max_{0 \leq |\alpha| \leq m} \|D^\alpha u\|_\infty$$

where $\|\cdot\|_p$ is the norm in $L^p(\Omega)$. The Sobolev spaces are consequently those on which $\|\cdot\|_{m,p}$ is a norm [12]:

- $H^{m,p}(\Omega) \equiv$ the completion of $\{u \in C^m(\Omega) : \|u\|_{m,p} < \infty\}$ with respect to the norm $\|\cdot\|_{m,p}$

- $W^{m,p}(\Omega) \equiv \{u \in L^p(\Omega) : D^\alpha u \in L^p(\Omega) \text{ for } 0 \leq |\alpha| \leq m\}$ where $D^\alpha u$ is the weak partial derivative
- $W_0^{m,p}(\Omega) \equiv$ the closure of C_0^∞ in the space $W^{m,p}(\Omega)$

It can be shown by Theorem 3.17 of [12] that for $1 \leq p < \infty$ we have

$$H^{m,p}(\Omega) = W^{m,p}(\Omega)$$

so for the rest of this section, we will use the space $H^{m,p}(\Omega)$.

5.1.3 Trace theory

5.1.4 Fractional Sobolev spaces

Suppose we have a general, possibly nonsmooth, open set $\Omega \subset \mathbb{R}^n$. For any real $s > 0$ and for any $p \in [1, \infty)$, we aim to define the fractional Sobolev spaces $W^{s,p}(\Omega)$.

To construct this, let's first fix the fractional exponent $s \in (0, 1)$. Then for any $p \in [1, +\infty)$ we define $W^{s,p}(\Omega)$ as [13]:

$$W^{s,p}(\Omega) := \left\{ u \in L^p(\Omega) : \frac{|u(x) - u(y)|}{|x - y|^{\frac{n}{p}+s}} \in L^p(\Omega \times \Omega) \right\}$$

which can be interpreted as an intermediary Banach space between $L^p(\Omega)$ and $W^{1,p}(\Omega)$ endowed with the norm

$$\|u\|_{W^{s,p}(\Omega)} := \left(\int_\Omega |u|^p \, \mathrm{d}x + \int_\Omega \int_\Omega \frac{|u(x) - u(y)|^p}{|x - y|^{n+sp}} \, \mathrm{d}x\mathrm{d}y \right)^{\frac{1}{p}}$$

where we have used the *Gagliardo seminorm* of u:

$$[u]_{W^{s,p}(\Omega)} := \left(\int_\Omega \int_\Omega \frac{|u(x) - u(y)|^p}{|x - y|^{n+sp}} \, \mathrm{d}x\mathrm{d}y \right)^{\frac{1}{p}}$$

5.2 Model

Suppose we would like to price an option on k assets $\boldsymbol{S} = (S^1, \ldots, S^k)$. Similar to before, these assets have theoretically unbounded prices, so we need to truncate it a minimum and maximum price in our model. Thus the domain of the target function can be chosen to be

$$\Omega = (S^1_{\text{min}}, S^1_{\text{max}}) \times \cdots \times (S^k_{\text{min}}, S^k_{\text{max}})$$

In our study we will make the simplification that $S^i_{\text{min}} = S^j_{\text{min}}$ and $S^i_{\text{max}} = S^j_{\text{max}}$ for all $i \neq j$, which we can call a and b respectively. Our spaces would consequently have the definitions

$$\Omega = (a, b)^n \qquad \Omega_T = (0, T) \times \Omega \qquad \Sigma = (0, T) \times \partial\Omega$$

where $\partial\Omega =: \Gamma$ is the boundary of Ω, a hyper-rectangle. We will use the representation

$$\Gamma = \bigcup_{i=1}^{n} \big(\{x \in [a,b]^n : x_i = a\} \cup \{x \in [a,b]^n : x_i = b\} \big)$$

which is not the simplest, but it is useful as it is decomposed into faces, which is useful when we would like to use the norm derivative later. Note that in the one-dimensional case, this reduces simply to the points $\{a, b\}$.

5.3 Loss function

Again, our loss function would be a weighted sum of four loss components, each of which ensures regulation of a specific aspect of our function.

1. The variational loss, which is as before, controlling the accuracy of our function in the interior of our domain:
$$J_1 = \| \min\{\mathcal{G}[f], f - g\} \|^2_{L^2(\Omega_T),\mu_1}$$

2. The energy loss: before we were only controlling the L2 norm of the payoff, but we require the option value to equal the payoff as a function and not just pointwise. Thus we need to incorporate the gradient into the loss so that the spatial slope of the payoff is respected.
$$J_2 = \|f(0,\cdot) - g(\cdot)\|^2_{H^1(\Omega),\mu_2}$$
where we have the norm defined as
$$\|d\|^2_{H^1(\Omega),\mu_2} = \|d\|^2_{L^2(\Omega)} + \|\nabla d\|^2_{L^2(\Omega)}$$
and the derivative
$$\nabla d = \big(\partial_{x_1} d, \ldots, \partial_{x_k} d\big)$$

3. J3 (need to explain intuitively)
$$J_3 = \|f(t,x) - g(x)\|^2_{H^{\frac{3}{4},\frac{3}{2}}(\Sigma),\mu_3}$$
where
$$\|d\|^2_{H^{\frac{3}{4},\frac{3}{2}}(\Sigma)} = \|d\|^2_{H^{0,1}(\Sigma)} + \int_0^T \int_0^T \frac{\|d(t_1,\cdot) - d(t_2,\cdot)\|^2_{L^2(\Gamma)}}{|t_1 - t_2|^{2.5}} \,\mathrm{d}t_1 \,\mathrm{d}t_2 + \int_0^T \int_\Gamma \int_\Gamma \frac{|d(t,x) - d(t,y)|^2}{\|x - y\|^2} \,\mathrm{d}\sigma(x)\mathrm{d}\sigma(y)\mathrm{d}t$$
The time exponent 2.5 is equal to $2p + 1$ where p is the decimal part of the time fractional exponent, and the space exponent 2 is equal to $2\theta + k - 1$ where θ is the decimal part of the space fractional exponent, and k the dimension. Note that here $\mathrm{d}\sigma(x)$ and $\mathrm{d}\sigma(y)$ indicate the surface measures on Γ, but in practice we can treat these as $\mathrm{d}x$ and $\mathrm{d}y$.

4. J4 (need to explain intuitively):
$$J_4 = \|\nabla(f - g) \cdot v\|^2_{H^{\frac{1}{4},\frac{1}{2}}(\Sigma),\mu_4}$$

where the norm derivative is $\nabla(f-g)\cdot\nu = \partial_{x_{i^\star}}(f-g)$ given that $x_{i^\star} \in \{a,b\}$ and $x_i \in (a,b)$ for $i \neq i^\star$. Rigorously, we require the sign but in practice this is not necessary, as we will square in the end. The fractional norm is thus written

$$\begin{aligned}\|d\|^2_{H^{\frac{1}{4},\frac{1}{2}}(\Sigma)} = \|d\|^2_{L^2(\Sigma)} &+ \int_0^T \int_0^T \frac{\|d(t_1,\cdot) - d(t_2,\cdot)\|^2_{L^2(\Gamma)}}{|t_1 - t_2|^{1.5}} \,\mathrm{d}t_1 \mathrm{d}t_2 \\ &+ \int_0^T \int_\Gamma \int_\Gamma \frac{|d(t,x) - d(t,y)|^2}{\|x-y\|^2} \,\mathrm{d}\sigma(x) \mathrm{d}(y) \mathrm{d}t\end{aligned}$$

because $H^{0,0} = L^2$.

In practice, we will compute a Monte Carlo estimate of these integrals by sampling pairs (t_1, t_2) from the domain, evaluating the integrand and taking the mean across all pairs. To see this, suppose we can sample iid a random variable X with probability density function $q(x)$. Then for any integrand $\phi(x)$ we have

$$\int \phi(x)\,\mathrm{d}x = \int \frac{\phi(x)}{q(x)} q(x)\,\mathrm{d}x = \mathbb{E}\left[\frac{\phi(X)}{q(X)}\right] \approx \frac{1}{N}\sum_{i=1}^{N} \frac{\phi(X_i)}{q(X_i)}$$

which can be generalised similarly to higher dimensions. By restricting our domain of t_1 and t_2 to be uniform in $[0,1]$, we have q uniformly equal to 1 and hence we can just evaluate the integrand.

5.4 Numerical results

Chapter 6

The Heston Model

The classic Black Scholes model makes the strong assumption that the stock returns are normally distributed with known mean and variance. However, the Black Scholes formula does not depend on the mean, so allowing the mean to vary does not help us to generalise the model. But if we allow volatility to vary according to a stochastic process, we obtain a more generalised model. Heston [14] proposes a model that is not based on the Black Scholes formula that provides a closed-form solution for European call options. In this model, the dynamics under the pricing measure of the asset price S and the variance (squared volatility) process V are governed by the stochastic differential equation system

$$\begin{aligned} \mathrm{d}S_t &= rS_t\mathrm{d}t + \sqrt{V_t}\mathrm{d}W_t^1 \\ \mathrm{d}V_t &= \kappa(\theta - V_t)\mathrm{d}t + \sigma\sqrt{V_t}\mathrm{d}W_t^2 \end{aligned}$$

where we have initial conditions $S_0 = s > 0$ and $V_0 = v \geq 0$, and the two Brownian motions are correlated with

$$\mathrm{d}\langle W^1, W^2\rangle_t = \rho\mathrm{d}t \qquad \rho \in (-1, 1)$$

The dynamics of the variance V follow a CIR process with mean reversion rate $\kappa > 0$, long run state $\theta > 0$ and volatility of the volatility $\sigma > 0$. We can enforce the Feller condition $2\kappa\theta \geq \sigma^2$ so that the volatility process stays positive.

6.1 Closed-form solution

There exists a closed-form solution derived by Heston [14] to the price of the European call option. This is done by using an Ansatz inspired by the Black-Scholes formula, and determining the elements of our guess by substituting it into the PDE we know the solution must satisfy along with the boundary conditions (derived in [reference]). We will include this here to serve as a benchmark for our other techniques. Of course, the other techniques will provide more flexibility such as early stopping for American options for which there does not exist a closed-form solution even for the Black Scholes model let alone the Heston model, but should the European prices coincide between the closed form solution and our other techniques, we can have more confidence that the American option prices are accurate.

For expiry $T > 0$, the solution takes the form

$$C(S, V, t) = SP_1 - Ke^{-r(T-t)}P_2$$

where

$$P_j = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Re\left(\frac{e^{-iu\ln K}\phi_j(u)}{iu}\right)\,\mathrm{d}u$$

for $j = 1, 2$. The characteristic equations $\phi_j(\cdot)$ have the solution

$$\phi_j(u) = \exp\left(C(T-t;u) + D(T-t;u)V + iu\ln S_0\right)$$

where

$$C(\tau;u) = rui\tau + \frac{a}{\sigma^2}\left\{(b_j - \rho\sigma ui + d)\tau - 2\ln\left(\frac{1-ge^{d\tau}}{1-g}\right)\right\}$$
$$D(\tau;u) = \frac{b_j - \rho\sigma ui + d}{\sigma^2}\left(\frac{1-e^{d\tau}}{1-ge^{d\tau}}\right)$$

and the quantities

$$g = \frac{b_j - \rho\sigma ui + d}{b_j - \rho\sigma ui - d}$$
$$d = \sqrt{(\rho\sigma ui - b_j)^2 - \sigma^2\big((3-2j)ui - u^2\big)}$$
$$a = \kappa\theta$$
$$b_1 = \kappa - \rho\sigma$$
$$b_2 = \kappa$$

These quantities are derived using Fourier Transforms and complex integration, refer to [15] for more details.

We can plot the call option prices against the stock price S_0. We expect it to be close to the Black Scholes formula and for it to collapse to the call option payoff at expiry as the option loses its time value. We see in Figure 6.1 that this behaviour is indeed exhibited.

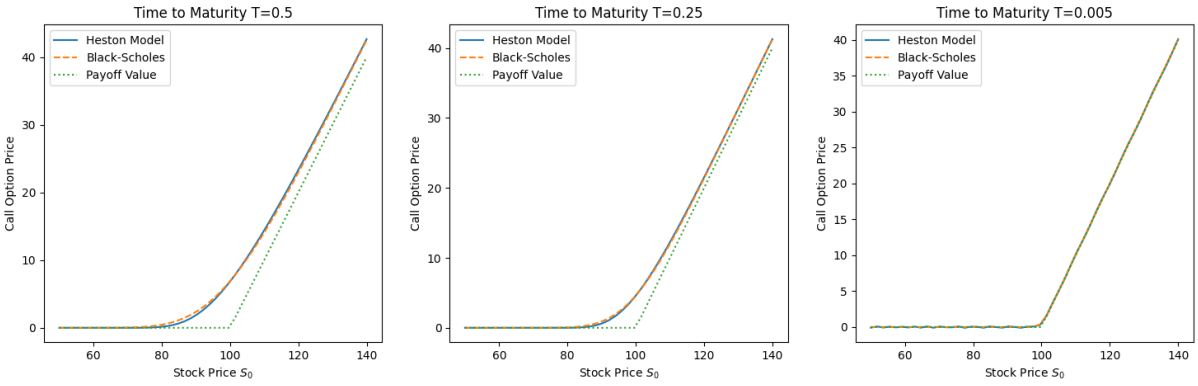

Figure 6.1 Call option values on different maturities

For European options, we can also use put call parity, which is model independent, to obtain the put prices:

$$\text{Call} - \text{Put} = S - Ke^{-rT}$$

where we can get the graphs with the same parameters as shown in Figure 6.2. As expected, we also get convergence towards the payoff at smaller time to maturities, and for greater time to maturities, we see that the value of the option is actually lower than the payoff, which is due to the interest rate.

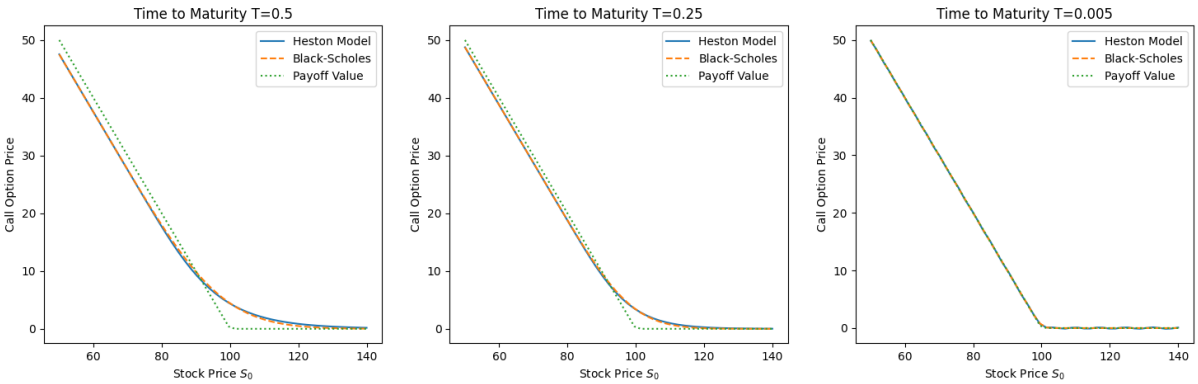

Figure 6.2 Put option values on different maturities

6.2 Tree Solution

Since the closed-form solution is limited to European options, we would like to find ways to approximate solutions to the pricing problem so that we can eventually extend the case to American options, which has no closed-form solution. The first method we will explore is the tree method, where it is easy to modify to the American case by adding an additional comparison to the intrinsic value at each node. We will use the closed-form formula as a benchmark for the tree method for European options, to give us confidence in the solution the tree model provides for American options as it is a simple modification in the algorithm, which we can use as a benchmark for the neural networks we will explore later.

The idea will be similar to the binomial tree we explored earlier in the paper, but we now have two correlated stochastic processes in the Heston model: the asset price process and the variance process. As this tree is not recombinant, generating paths for these processes and computing the option prices by backwards induction naively would lead to an exponential time complexity, which is not computationally feasible. Therefore, we will use a grid interpolation approach as in [16] so that the number of nodes grows quadratically in the number of time steps instead.

6.2.1 Algorithm

For the algorithm, we will work under the variance process V and log stock price process Z (which is a simple transformation from S using Itô's formula):

$$\begin{aligned} \mathrm{d}V_t &= \kappa(\theta - V_t)\mathrm{d}t + \sigma\sqrt{V_t}\mathrm{d}W_t^1 \\ \mathrm{d}Z_t &= (r - \tfrac{1}{2}V_t)\mathrm{d}t + \sqrt{V_t}\mathrm{d}W_t^2 \end{aligned}$$

where $\mathrm{d}\langle W^1, W^2\rangle_t = \rho$ under the risk-neutral measure $\mathbb{Q}$ and $\kappa, \sigma, \theta, r > 0$ as before. Of course, the value C of the European option with maturity T and payoff function in terms of stock and volatility values $g : \mathbb{R}^+ \times \mathbb{R}^+ \to \mathbb{R}$ will be

$$C(t, V_t, Z_t) = e^{-r(T-t)}\mathbb{E}^{\mathbb{Q}}[g(e^{Z_t}, \sqrt{V_t}) \mid \mathcal{F}_t]$$

Euler Scheme

To construct the tree, we need to discretise the process. Suppose we have n steps, then $\Delta t = \frac{T}{n}$. Using the standard Euler scheme, we can have

$$V_{k+1} = V_k + \kappa(\theta - V_k^+)\Delta t + Y_{k+1}^1 \sigma \sqrt{V_k^+ \Delta t}$$
$$Z_{k+1} = Z_k + (r - \tfrac{1}{2}V_k)\Delta t + Y_{k+1}^2 \sqrt{V_k^+ \Delta t}$$

where the increment variables (Y_k^1, Y_k^2) are iid in k with probabilities

$$\mathbb{Q}^n(Y_k^1 = i, Y_k^2 = j) = \frac{1}{4}(1 + ij\rho) \qquad i, j \in \{-1, 1\}$$

We use a different measure $\mathbb{Q}^n$ as it is not the same as the risk-neutral measure in the Heston model. To ensure positivity of the process, we have taken the positive part of the variance process $V_k^+ := \max(0, V_k)$.

Interpolation optimisation

The problem with the naïve Euler scheme is that the tree is not recombining, and hence the number of nodes scales exponentially with increasing time steps, which is not feasible computationally. We will hence, for each time step k, construct a grid of option prices at time k and the backward induction would be a bilinear interpolation of the four nearest points.

To set this up, we need the boundaries of the grid at each time step k, which is given by the minimum and maximum values of V_k and Z_k in the support. In other words, we have

$$z_k^{\max} = \max\{z : \mathbb{Q}^n(Z_k = z) > 0\}$$
$$z_k^{\min} = \min\{z : \mathbb{Q}^n(Z_k = z) > 0\}$$
$$v_k^{\max} = \max\{v : \mathbb{Q}^n(V_k = v) > 0\}$$
$$v_k^{\min} = \min\{v : \mathbb{Q}^n(V_k = v) > 0\}$$

With these boundaries, we have the grid spacing given our mesh sizes $m_z, m_v \in \mathbb{N}^+$:

$$\Delta z_k = (z_k^{\max} - z_k^{\min})/m_z$$
$$\Delta v_k = (v_k^{\max} - v_k^{\min})/m_v$$

and the gridpoints:

$$\hat{\mathcal{S}}_k = \left\{ \left(v_k^{\min} + i\Delta v_k, z_k^{\min} + j\Delta z_k \right) \mid i \in \{0, \dots, m_v\}, j \in \{0, \dots, m_z\} \right\}$$

We will now define the discretised process $(\tilde{V}, \tilde{Z})$ on the grid space $\hat{\mathcal{S}}_k$. This will involve the Euler scheme functions

$$\nu(y, v, z) = v + \kappa(\theta - v^+)\Delta t + y\sigma\sqrt{v^+ \Delta t}$$
$$\zeta(y, v, z) = z + (r - \tfrac{1}{2}v)\Delta t + y\sqrt{v^+ \Delta t}$$

Then using these functions, we can retrieve the next step. We can evolve our process to the next step with

$$(\tilde{V}_{k+1}, \tilde{Z}_{k+1}) = \text{snap}^{\hat{\mathcal{S}}_{k+1}}_{Y_{k+1}^3, Y_{k+1}^4} \left(\nu(Y_{k+1}^1, \tilde{V}_k, \tilde{Z}_k), \zeta(Y_{k+1}^1, \tilde{V}_k, \tilde{Z}_k) \right)$$

Where $\text{snap}^{\mathcal{S}}_{i_3,i_4} : \mathbb{R}^2 \to \mathcal{S} \subset \mathbb{R}^2$ is a function mapping $(v, z) \in \mathbb{R}^2$ to one of the grid points in $\mathcal{S} := \mathcal{S}_v \times \mathcal{S}_z$. $i_3, i_4 \in \{0, 1\}$ are additional parameters that determine the corner in which (v, z) is mapped.

To do this, we define the 1D snapping functions $\phi^{\mathcal{T}}_i : \mathbb{R} \to \mathcal{T}$ where we have the one-dimensional grid points $\mathcal{T} \subset \mathbb{R}$ and the snap-right indicator $i \in \{0, 1\}$. The function is defined mathematically as

$$\phi^{\mathcal{T}}_0(u) = \max\{u' \in \mathcal{T} : u' \leq u\}$$
$$\phi^{\mathcal{T}}_1(u) = \min\{u' \in \mathcal{T} : u' \geq u\}$$

Our 2D snapping function $\text{snap}^{\mathcal{S}}_{i_3,i_4} : \mathbb{R}^2 \to \mathcal{S}$ where $\mathcal{S} = \mathcal{S}_v \times \mathcal{S}_z \subset \mathbb{R}^2$ is thus

$$(v, z) \mapsto \left(\phi^{\mathcal{S}_v}_{i_3}(v), \phi^{\mathcal{S}_z}_{i_4}(z)\right)$$

The joint probabilities [16] for Y_{k+1} is given by, for $i_1, i_2 \in \{-1, 1\}, i_3, i_4 \in \{0, 1\}$:

$$\begin{aligned} q_{i_1,i_2,i_3,i_4}(\tilde{V}_k, \tilde{Z}_k) &:= \tilde{\mathbb{Q}}(Y^1_{k+1} = i_1, Y^2_{k+1} = i_2, Y^3_{k+1} = i_3, Y^4_{k+1} = i_4 \mid (\tilde{V}_k, \tilde{Z}_k)) \\ &= \frac{1}{4}(1 + i_1 i_2 \rho) \times c^{\hat{\mathcal{S}}_{k+1}}_{i_3 i_4}\left(\nu(i_1, \tilde{V}_k, \tilde{Z}_k), \zeta(i_2, \tilde{V}_k, \tilde{Z}_k)\right) \end{aligned}$$

Here we have used the interpolation quantity [17], where on a general grid set $\mathcal{S} = S_v \times S_z \subset \mathbb{R}^2$ we have

$$c^{\mathcal{S}}_{i_3 i_4}(v, z) = \begin{cases} (1 - \tilde{v})(1 - \tilde{z}) & (i_3, i_4) = (0, 0) \\ \tilde{v}(1 - \tilde{z}) & (i_3, i_4) = (1, 0) \\ (1 - \tilde{v})\tilde{z} & (i_3, i_4) = (0, 1) \\ \tilde{v}\tilde{z} & (i_3, i_4) = (1, 1) \end{cases}$$

where we have the difference terms

$$\tilde{v} = \frac{v - \phi^{\mathcal{S}_v}_0(v)}{\phi^{\mathcal{S}_v}_1(v) - \phi^{\mathcal{S}_v}_0(v)} = \frac{v - \phi^{\mathcal{S}_v}_0(v)}{\Delta v_k}$$
$$\tilde{z} = \frac{z - \phi^{\mathcal{S}_z}_0(z)}{\phi^{\mathcal{S}_z}_1(z) - \phi^{\mathcal{S}_z}_0(z)} = \frac{z - \phi^{\mathcal{S}_z}_0(z)}{\Delta z_k}$$

because our grid is evenly spaced at all time steps k.

Backwards induction

We can now work backwards on the tree, having constructed the grids. Our base case is the final grid, where the price is just the payoff as we are at maturity. For a general time t_k at step k and node $(v, z) \in \hat{\mathcal{S}}_k$, we will generate the four possible steps determined by y_1, y_2 each taking values in $\{-1, 1\}$, and evolve our node with

$$(v, z) \mapsto \left(\nu(y_1, v, z), \zeta(y_2, v, z)\right) =: (v', z')$$

Which will be in between four points on the grid $\hat{\mathcal{S}}_{k+1}$. These can be found by performing the snap function on our evolved node: for $y_3, y_4 \in \{0, 1\}$ we can encapsulate $y = (y_1, y_2, y_3, y_4)$ and we have

$$(v_y, z_y) = \text{snap}^{\hat{\mathcal{S}}_{k+1}}_{y_3,y_4}(v', z')$$

Each of these will have a known option price. Therefore, we can just compute the discounted expectation of these sixteen nodes (two choices for each of $y_1, \ldots, y_4$) for the option price at the original node:

$$C(t_k, v, z) = e^{-r\Delta t} \sum_{y \in \mathcal{Y}} q_{y_1,y_2,y_3,y_4}(v, z) \times C(t_{k+1}, v_y, z_y)$$

where $\mathcal{Y} = \{-1, 1\}^2 \times \{0, 1\}^2$.

Implementation techniques

If we implemented this method naively, it would be very difficult for us to use the binomial tree as a benchmark because it would need to construct a new tree from scratch for every single value of S_0, V_0 and T. Thus any heatmaps or plots we create would require hundreds of trees which is not feasible, considering each tree requires many time steps and large mesh sizes to be accurate.

Thus in addition to vectorising our code for each tree, we will instead consider an initial range $[S_0^{\min}, S_0^{\max}]$ and $[V_0^{\min}, V_0^{\max}]$ which we will use to construct our grid bounds. More specifically, $z_k^{\max}$ and $v_k^{\max}$ will be the up-most path starting with $(S_0^{\max}, V_0^{\max})$ instead of simply some individual value (S_0, V_0), and similar for the down paths. In this way, when we use the binomial tree as a benchmark, we can use any initial value of (S_0, V_0) as long as it's in the original range and compute the price with an interpolation as described above. With this method, we can evaluate across different initial values while constructing just one tree. We sacrifice a little accuracy in doing so because the grids would be more widely spaced, which we can combat by increasing the mesh sizes. Thus, we effectively construct one more complex tree instead of hundreds of simpler ones, which is much more computationally efficient.

6.2.2 Numerical Results

It can be shown that if we have

$$\lim_{n\to\infty} \Delta t = 0, \qquad \lim_{n\to\infty} \max_{k=1,\ldots,n-1} \frac{\Delta v_k^n}{\Delta t_k} = 0, \qquad \lim_{n\to\infty} \max_{k=1,\ldots,n-1} \frac{\Delta z_k^n}{\Delta t_k} = 0$$

then the process represented in our Heston tree converges weakly to the original process (V, Z) [16]. Note here we denote Δv_k from before as Δv_k^n to emphasize we have split our maturity T into n steps, and similarly for Δz_k^n.

Taking this into account, we choose our parameters to be $n = 100, m_v = 300, m_z = 600$ which allows our tree to faithfully reproduce the closed form solution while keeping runtime low (after I optimised the algorithm with Python vectorisation). Figure 6.3 are the results after using the parameters $K = 1, T = 1, r = 0.05, \kappa = 2, \theta = 0.04, \sigma = 0.3, \rho = -0.7$. We produce these plots by varying across different values of S_0 (for delta), v_0 (for vega) and time elapsed (for term structure), and all else kept constant.

We can see that the Heston tree does indeed replicate the closed form solution very closely to very small error. This gives us confidence in our implementation, and with an easy modification we can compute the American option prices which we will use as a benchmark.

To compute the American option price with the tree model, all we need to do is to introduce early stopping at every node: we evaluate the intrinsic value the option has at a node and set the option value to be the maximum of the intrinsic value and the discounted continuation value.

Figure 6.3 Call option prices by the Heston Tree

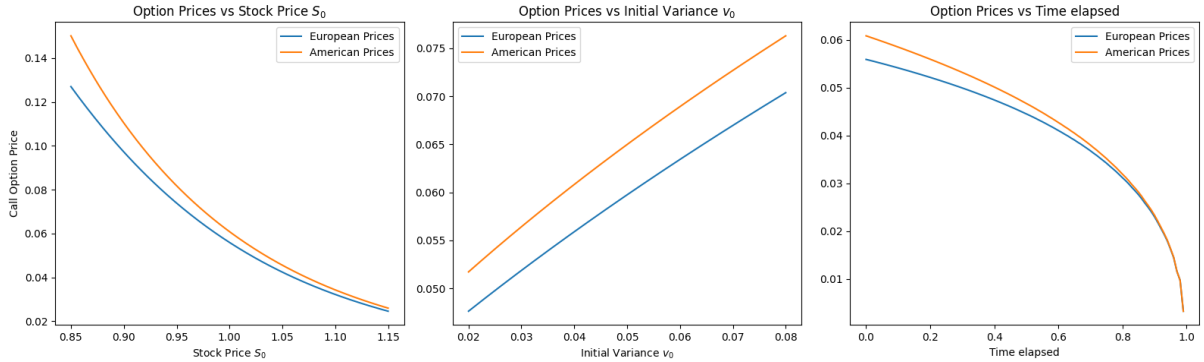

Figure 6.4 American vs European option prices for the put

In Figure 6.4 we can see that the European option price indeed bounds the American prices from below, as we would expect. We have confidence that our American prices are accurate since our binomial tree faithfully approximates the closed-form solution for European options. We will now use this as a benchmark for neural network approximations.

6.3 Neural Network Solution

6.3.1 Variational Equation

We would now like to derive the variational equation for options on assets under the Heston model. Note that for a general process $X_t \in \mathbb{R}^d$ solving the Itô SDE

$$\mathrm{d}X_t = b(X_t)\mathrm{d}t + \sigma(X_t)\mathrm{d}W_t$$

where we have the drift $b : \mathbb{R}^d \to \mathbb{R}^d$, the volatility $\sigma : \mathbb{R}^d \to \mathbb{R}^{d\times m}$ and the m-dimensional Brownian motion W_t. On second-order smooth functions $f \in C^2(\mathbb{R}^d)$, the infinitesimal generator $\mathcal{L}$ is defined as [18]

$$\mathcal{L}f(x) := \lim_{t\to 0} \frac{\mathbb{E}[f(X_t) \mid X_0 = x] - f(x)}{t}$$

which exists and can be derived with Itô's formula (or Taylor's expansion) to be [19]

$$\mathcal{L}f(x) = \sum_{i=1}^{d} b_i(x)\frac{\partial f}{\partial x_i} + \frac{1}{2}\sum_{i=1}^{d}\sum_{j=1}^{d}\left(\sigma\sigma^\top\right)_{ij}(x)\frac{\partial^2 f}{\partial x_i \partial x_j}$$

Let's apply this to our Heston model. The drift vector is

$$b(S,V) = \begin{pmatrix} rS \\ \kappa(\theta - V) \end{pmatrix}$$

To compute $\sigma\sigma^\top$, we need to represent our correlated Brownian motions in terms of independent ones. Suppose we have independent Brownian motions B^1 and B^2, we can define

$$\begin{aligned} W^1 &= B^1 \\ W^2 &= \rho B^1 + \sqrt{1-\rho^2} B^2 \end{aligned}$$

which has the quadratic variation $\mathrm{d}\langle W^1, W^2\rangle_t = \rho \mathrm{d}t$ as required. Using these new Brownian motions, we can rewrite the SDE as

$$\mathrm{d}\begin{pmatrix} S_t \\ V_t \end{pmatrix} = \underbrace{\begin{pmatrix} rS_t \\ \kappa(\theta - V_t) \end{pmatrix}}_{b(S_t,V_t)} \mathrm{d}t + \underbrace{\begin{pmatrix} S_t\sqrt{V_t} & 0 \\ \rho\sigma\sqrt{V_t} & \sigma\sqrt{V_t}\sqrt{1-\rho^2} \end{pmatrix}}_{\sigma(S_t,V_t)} \begin{pmatrix} \mathrm{d}B_t^1 \\ \mathrm{d}B_t^2 \end{pmatrix}$$

which means that our covariance matrix is now

$$\sigma(S,V) = \begin{pmatrix} S\sqrt{V} & 0 \\ \rho\sigma\sqrt{V} & \sigma\sqrt{V}\sqrt{1-\rho^2} \end{pmatrix}.$$

which means

$$\sigma\sigma^\top = \begin{pmatrix} S^2V & \rho\sigma SV \\ \rho\sigma SV & \sigma^2 V \end{pmatrix}$$

We now have everything we need for the formula. For any function $f = f(S,V) \in C^2(\mathbb{R}^d)$, we have

$$\mathcal{L}f = rS\frac{\partial f}{\partial S} + \kappa(\theta - V_t)\frac{\partial f}{\partial V} + \frac{1}{2}\left(S^2V\frac{\partial^2 f}{\partial S^2} + 2\rho\sigma SV\frac{\partial^2 f}{\partial S \partial V} + \sigma^2 V\frac{\partial^2 f}{\partial V^2}\right)$$

This is our infinitesimal generator $\mathcal{L}$ acting on smooth functions f. Suppose we have a European option with payoff $g : \mathbb{R}^+ \to \mathbb{R}^+$, then our price function $u(t,S,V)$, the value of the call option with expiry T at time t with $S_t = S$ and $V_t = V$ satisfies

$$\mathcal{G}[u] := ru - \frac{\partial u}{\partial t} + \mathcal{L}u = 0$$

with terminal condition $u(T,S,V) = g(S)$.

As a check, we can set $\kappa = 0$ and $\sigma = 0$ so that $\mathrm{d}V = 0$ and volatility becomes a constant, and our model collapses to what we have been working with before. Our generator becomes

$$\mathcal{L}f = rS\frac{\partial f}{\partial S} + \frac{1}{2}S^2 v_0 \frac{\partial^2 f}{\partial S^2}$$

which matches our expression from before, since v_0 is the initial variance.

We have derived the variational equation for the European option for simplicity, but it can be easily extended for the American case. With the same argument as 4.1, the value of the American option must satisfy the PDE

$$\min\big(\mathcal{G}[u], v(t,S,V) - g(S)\big) = 0$$

6.3.2 Loss Function

To use a neural network to approximate the function $u(t, S, V)$, we need to establish the relevant boundary conditions. This will help us ensure regularity in all parts of the domain, and we will also incorporate the first derivative to ensure that our neural network approximation also captures the behaviour of the delta (derivative of option price with respect to stock price). The boundary conditions for the call option are provided in [14], but we will adapt them for put options in our study, as they exhibit different behaviour for American options:

1. $t \to T$: the value at expiry is just the intrinsic value, or the payoff of the put option:

$$u(T, S, V) = \max(0, K - S)$$

2. $S \to 0$: the option value is the strike when the stock is worthless, because we can exercise immediately (for European options we would require a discounting term):

$$u(t, 0, V) = K$$

3. $S \to \infty$: the value of a very far out of the money option is 0, as it is very unlikely for it to expire in of the money:

$$u(t, \infty, V) = 0$$

4. $V \to 0$: when the volatility is 0, our PDE collapses to

$$rS\frac{\partial u}{\partial S}(t, S, 0) + \kappa\theta\frac{\partial u}{\partial V}(t, S, 0) - ru(t, S, 0) + \frac{\partial u}{\partial t}(t, S, 0) = 0$$

5. $V \to \infty$: when the volatility is very high, it is almost sure that the product of assets would be zero at some point before expiry, so the value of the option is the strike:

$$u(t, S, \infty) = K$$

In practice, as we clearly cannot represent infinity, we will settle for large constants for the asset price S_{inf} and variance V_{inf}. The minimum for both of these will be taken to be zero. The loss components for our neural network can thus be directly translated. Again, for each epoch, we will take a batch of size B samples for (t, S, V) from our domain

$$\Omega = (0, T) \times (0, S_{\text{max}}) \times (0, V_{\text{max}})$$

and compute the mean squared error like before, where S_{max} and V_{max} are chosen constants for our pricing range. Mathematically, let the function represented by the neural network be f. For epoch i, we have the samples $\left(t_1^{(i)}, \ldots, t_B^{(i)}\right)$, $\left(S_1^{(i)}, \ldots, S_B^{(i)}\right)$ and $\left(V_1^{(i)}, \ldots, V_B^{(i)}\right)$, for which we

can compute the loss components

$$J_1^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\left\|\min\left\{\mathcal{G}\left[f\left(t_j^{(i)}, S_j^{(i)}, V_j^{(j)}\right)\right], f\left(t_j^{(i)}, S_j^{(i)}, V_j^{(j)}\right) - \max\left(0, K - S_j^{(i)}\right)\right\}\right\|_2^2$$

$$J_2^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\left\|f\left(T, S_j^{(i)}, V_j^{(i)}\right) - \max\left(0, K - S_j^{(i)}\right)\right\|_2^2$$

$$J_3^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\left\|f\left(t_j^{(i)}, 0, V_j^{(i)}\right) - K\right\|_2^2$$

$$J_4^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\left\|f\left(t_j^{(i)}, S_{\text{inf}}, V_j^{(i)}\right)\right\|_2^2$$

$$J_5^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\left\|rS_j^{(i)}\frac{\partial f}{\partial S}(t_j^{(i)}, S_j^{(i)}, 0) + \kappa\theta\frac{\partial f}{\partial V}(t_j^{(i)}, S_j^{(i)}, 0) - rf(t_j^{(i)}, S_j^{(i)}, 0) + \frac{\partial f}{\partial t}(t_j^{(i)}, S_j^{(i)}, 0)\right\|_2^2$$

$$J_6^{(i)} = \frac{1}{B}\sum_{j=1}^{B}\left\|f\left(t_j^{(i)}, S_j^{(i)}, V_{\text{inf}}\right)\right\|_2^2$$

where our final loss will be, again, a convex combination of the six loss components. In other words, for $\boldsymbol{\lambda} \in \Delta_6$, the final loss is

$$J^{(i)} = \boldsymbol{\lambda}^\top \boldsymbol{J}^{(i)} = \sum_{m=1}^{6} \lambda_m J_m^{(i)}$$

6.3.3 Numerical Results

We are now ready to train the neural network to price our option. We choose to use a learning rate of 1×10^{-3} with the Adam optimiser and the Mish activation function. By using parameters $S_0 = 1, K = 1, T = 1, r = 0.05, \kappa = 2, \theta = 0.04, \sigma = 0.3, \rho = -0.7$ obtain the plots as shown in Figure 6.5 for the American put.

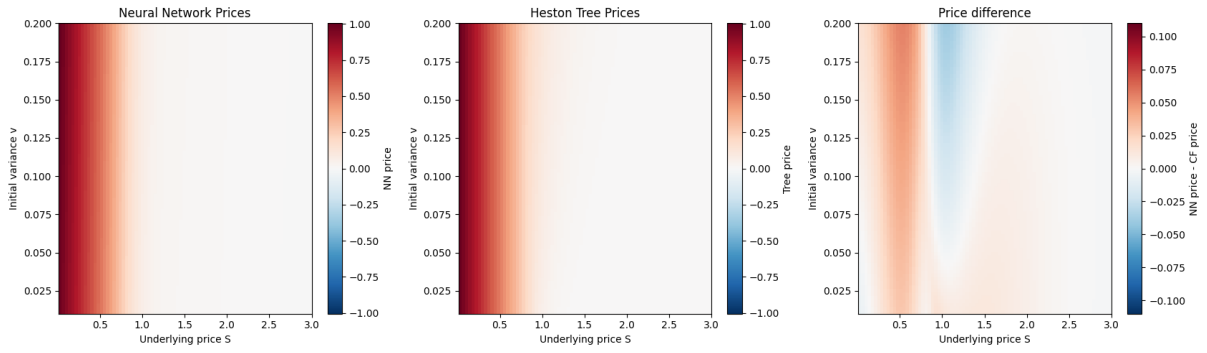

Figure 6.5 American put option prices

We can see that the neural network indeed faithfully reproduces the solution approximated by the binomial tree, even though it tends to perform less well at the money especially at greater volatilities. This can also be easily shown by fixing a volatility and plotting the approximated option value against the underlying, as shown in in Figure 6.6.

We can see that the tree prices are indeed well approximated by the Neural network, but the at-the-money options are underpriced for higher volatilities. This is due to the lack of a strict

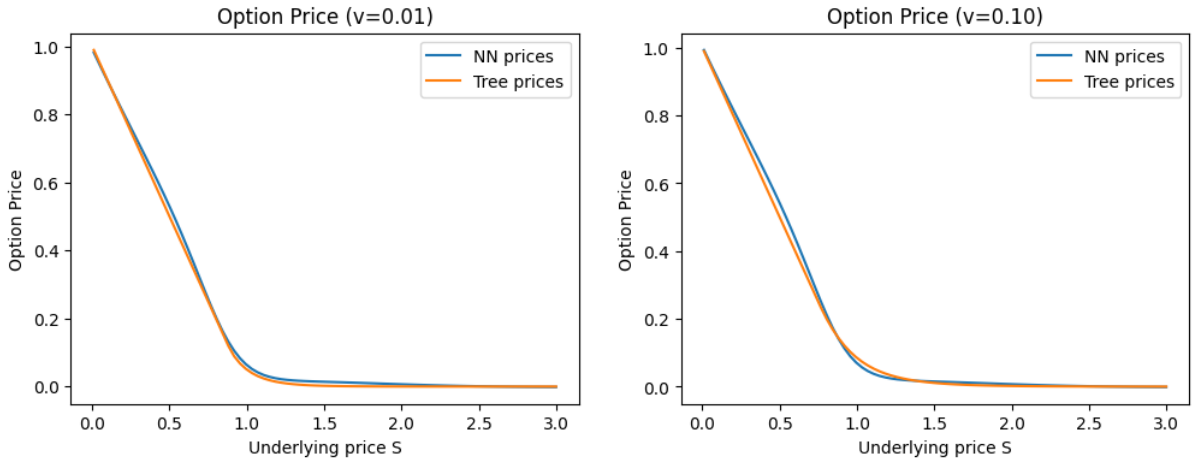

Figure 6.6 American put option prices with volatility fixed

high initial variance boundary condition. We can plot the option prices with respect to initial variance in Figure 6.7. Our price does indeed increase with increasing initial variance, but not at the same rate.

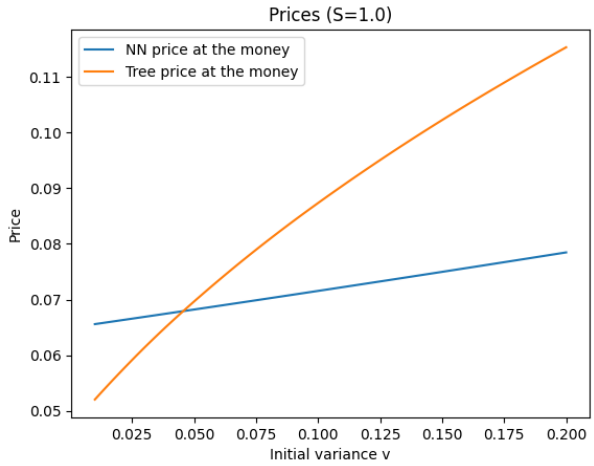

Figure 6.7 At-the-money price with varying initial variance

6.4 Multiple assets

With the neural network established, we are now ready to move on to pricing options on multiple assets. As with the Black-Scholes case, we will price the put on the product of multiple assets because we can find a one-dimensional reduction. Here, we will consider a simplified model. Suppose we have n assets and our assets $S = (S_1, \ldots, S_n)^\top$ are governed by the stochastic differential equations:

$$\begin{aligned} \mathrm{d}S &= \mathrm{diag}(S)(r\,\mathrm{d}t + \sqrt{V}\sigma\Sigma\,\mathrm{d}W_1) \\ \mathrm{d}V &= \kappa(\theta - V)\,\mathrm{d}t + \bar{\sigma}\sqrt{V}\,\mathrm{d}W_2 \end{aligned}$$

where we have the vector of iid Brownian motions $W_1 = (W_{1,1}, \ldots, W_{1,n})^\top$, variance vector $\sigma = (\sigma_1, \ldots, \sigma_n)^\top$ and the covariance matrix Σ, where for simplicity we will assume all cross-

correlations $\Sigma_{ij} = \rho$ for $i \neq j$. Assume additionally that each asset is correlated with the variance process: $\mathrm{d}W_{1,j}\mathrm{d}W_2 = \rho_j \mathrm{d}t$. Note that this means the asset S_i satisfies

$$\mathrm{d}S_i = S_i \left(r\,\mathrm{d}t + \sqrt{V}\sigma_i \sum_{j=1}^{n} \Sigma_{ij}\,\mathrm{d}W_{1,j} \right)$$

To guarantee positiveness of the variance process, we will enforce the Feller condition on the parameters

$$2\kappa\theta \geq \bar{\sigma}^2$$

We make these assumptions to allow a one-dimensional reduction for the Heston model, which would provide us a comparable benchmark for our results with the tree method. This would in turn give us confidence for our approximated solutions, should we wish to relax assumptions or to change the payoff.

6.4.1 One-dimensional reduction

Since we would like to find an equivalent formulation of the product of assets $X := \prod_{i=1}^{n} S_i$, it would be easier to work in the log domain. That is, find the governing SDE for the log process $Y = \log X = \sum_{i=1}^{n} \log S_i$ and then exponentiate with Itô's lemma to recover the desired result.

Let $Y_i = \log S_i$. Then by Itô's lemma, we have

$$\begin{aligned}
\mathrm{d}Y_i &= \frac{1}{S_i}\,\mathrm{d}S_i - \frac{1}{2S_i}\,\mathrm{d}S_i\,\mathrm{d}S_i \\
&= r\,\mathrm{d}t + \sqrt{V}\sigma_i \sum_{j=1}^{n} \Sigma_{ij}\,\mathrm{d}W_{1,j} - \frac{1}{2}V\sigma_i^2\,\mathrm{d}t
\end{aligned}$$

because cross-correlations are zero within W_1. We can collect terms to get

$$\mathrm{d}Y = \left(nr - \frac{1}{2}V\sum_{i=1}^{n}\sigma_i^2 \right)\mathrm{d}t + \sqrt{V}\sum_{j=1}^{n}\sum_{i=1}^{n}\sigma_i\Sigma_{ij}\,\mathrm{d}W_{1,j}$$

For ease of notation, let $A = \sum_{i=1}^{n}\sigma_i^2$ and $b_j = \sum_{i=1}^{n}\sigma_i\Sigma_{ij}$. Then we have

$$\mathrm{d}Y = \left(nr - \frac{1}{2}VA \right)\mathrm{d}t + \sqrt{V}\sum_{j=1}^{n} b_j\,\mathrm{d}W_{1,j}$$

Now Y has variance

$$\mathrm{d}Y\,\mathrm{d}Y = V\sum_{j=1}^{n} b_j^2\,\mathrm{d}t =: VB\,\mathrm{d}t$$

where we let $B := \sum_{j=1}^{n} b_j^2$ for convenience, we can rewrite $\mathrm{d}Y$ with a new Brownian motion:

$$\mathrm{d}Y = \left(nr - \frac{1}{2}VA \right)\mathrm{d}t + \sqrt{VB}\,\mathrm{d}W_X$$

where we have

$$\mathrm{d}W_X = \frac{\sum_{j=1}^{n} b_j\,\mathrm{d}W_{1,j}}{\sqrt{B}}$$

Using our correlation structure, we have

$$b_j = \sigma_j + \rho \sum_{i \neq j} \sigma_i = (1-\rho)\sigma_j + \rho \underbrace{\sum_{i=1}^{n} \sigma_i}_{=:S_\sigma}$$

So the sum of squares B is just

$$\begin{aligned} B = \sum_{j=1}^{n} b_j^2 &= \sum_{j=1}^{n} \left((1-\rho)\sigma_j + \rho S_\sigma\right)^2 \\ &= (1-\rho)^2 A + S_\sigma^2 (2\rho(1-\rho) + n\rho^2) \end{aligned}$$

We now have all components to exponentiate to recover $X = e^Y$. Using Itô's lemma again, we have

$$\begin{aligned} \mathrm{d}X &= X\,\mathrm{d}Y + \frac{1}{2} X\,\mathrm{d}Y\,\mathrm{d}Y \\ &= X\left(nr + \frac{1}{2}V(B-A)\right)\mathrm{d}t + X\sqrt{VB}\,\mathrm{d}W_X \end{aligned}$$

This is close to the Heston model, with the diffusion $\tilde{V} = VB$ which gives us

$$\begin{aligned} \mathrm{d}\tilde{V} &= B\,\mathrm{d}V \\ &= \kappa(B\theta - \tilde{V})\,\mathrm{d}t + \bar{\sigma}\sqrt{B}\sqrt{\tilde{V}}\,\mathrm{d}W_2 \end{aligned}$$

which is a new CIR process $\tilde{V}$ with parameters $\tilde{\kappa} = \kappa$, $\tilde{\theta} = B\theta$ and $\tilde{\sigma} = \bar{\sigma}\sqrt{B}$. The product of assets nearly gives a direct reduction to a one-dimensional Heston process, but the drift of X contains V which is a stochastic process itself. Thus for simplicity and for our benchmark, we will set $\rho = 0$ which means $B = A$ and the drift term becomes deterministic. As such, we are able to recover a one-dimensional Heston model for the product of assets X with variance $\tilde{V}$ and drift nr.

6.4.2 Variational Equation

Recall for a process $X_t \in \mathbb{R}^d$ solving $\mathrm{d}X_t = b(X_t)\mathrm{d}t + \sigma(X_t)\mathrm{d}W_t$, the infinitesimal generator $\mathcal{L}$ can be derived with Itô's formula [19] on $f(x)$ to be

$$\mathcal{L}f(x) = \sum_{i=1}^{d} b_i(x)\frac{\partial f}{\partial x_i} + \frac{1}{2}\sum_{i=1}^{d}\sum_{j=1}^{d}\left(\sigma\sigma^\top\right)_{ij}(x)\frac{\partial^2 f}{\partial x_i \partial x_j}$$

In our case, we have a function $f = f(S_1, \ldots, S_d, V)$, and we can derive the infinitesimal generator much like the 1D case, but this time we need to sum over our different asset and variance processes. Mathematically, our generator can be expressed to be

$$\begin{aligned} \mathcal{L}f = &\sum_{i=1}^{d} rS_i\frac{\partial f}{\partial S_i} + \kappa(\theta - V)\frac{\partial f}{\partial V} + \frac{1}{2}\sum_{i=1}^{d}\sum_{j=1}^{d}\Sigma_{ij}VS_iS_j\frac{\partial^2 f}{\partial S_i \partial S_j} \\ &+ \frac{1}{2}\bar{\sigma}^2 V\frac{\partial^2 f}{\partial V^2} + \sum_{i=1}^{d}\rho_i\sigma_i\bar{\sigma}V_iS_i\frac{\partial^2 f}{\partial S_i \partial V} \end{aligned}$$

For any payoff $g(\cdot)$, we have, as before, the variational equation for an European option being

$$\mathcal{G}[f] := rf - \frac{\partial f}{\partial t} + \mathcal{L}u = 0$$

and for an American option, we can introduce the early stopping criterion and the variation equation becomes

$$\min(\mathcal{G}[f], f - g) = 0$$

6.4.3 Loss function

Similar to what we did in the Black-Scholes model, at each epoch we will sample B points to approximate the loss components in each epoch. We would sample for the variance process, asset prices and time elapsed. Introduce our samples, for $\boldsymbol{S} = (S_1, \ldots, S_n)$:

$$\begin{aligned} t_1, \ldots, t_B &\in [0, T]^B \\ V_1, \ldots, V_B &\in (V_{\text{min}}, V_{\text{max}}) \\ \boldsymbol{S}_1, \ldots, \boldsymbol{S}_B &\in (S^1_{\text{min}}, S^1_{\text{max}}) \times \cdots \times (S^n_{\text{min}}, S^n_{\text{max}}) \end{aligned}$$

where $S^k_{\text{min}}, S^k_{\text{max}}$ is the modelled minimum and maximum prices asset k could reach, similar for V_{min} and V_{max}. Of course the variational loss for the American put needs to be included: in epoch i, we have

$$J_1^{(i)} = \frac{1}{B}\sum_{j=1}^{B} \left\| \min\left\{ \mathcal{G}\left[f\left(t_j^{(i)}, \boldsymbol{S}_j^{(i)}, V_j^{(j)}\right)\right], f\left(t_j^{(i)}, \boldsymbol{S}_j^{(i)}, V_j^{(j)}\right) - \max\left(0, K - \boldsymbol{S}_j^{(i)}\right)\right\}\right\|_2^2$$

We will use the same principle ideas for the boundary conditions as the one-dimensional case:

1. $t \to T$: the value at expiry is just the intrinsic value which is the payoff:

   $$u(T, S_1, \ldots, S_n, V) = \max\left(0, K - \prod_{k=1}^{n} S_k\right)$$

   which means that the loss would be

   $$J_2^{(i)} = \frac{1}{B}\sum_{j=1}^{B} \left\| f\left(T, \boldsymbol{S}_j^{(i)}, V_j^{(i)}\right) - \max\left(0, K - \boldsymbol{S}_j^{(i)}\right)\right\|_2^2$$

2. $S \to 0$: if one of the assets reaches zero, then the product of assets is consequently zero and we should exercise immediately. Thus the value is the strike. To implement this, let $\tilde{\boldsymbol{S}}_j^{(i)}$ be $\boldsymbol{S}_j^{(i)}$ with one of the assets modified manually to 0. Then this loss component is

   $$J_3^{(i)} = \frac{1}{B}\sum_{j=1}^{B} \left\| f\left(t_j^{(i)}, \tilde{\boldsymbol{S}}_j{}^{(i)}, V_j^{(i)}\right) - K\right\|_2^2$$

3. $S \to \infty$: if one of the assets has a very high price, then the product of assets would also be very high meaning the option would expire worthless out of the money. To implement this, let $\bar{\boldsymbol{S}}_j^{(i)}$ be $\boldsymbol{S}_j^{(i)}$ with one of the assets modified manually to S_{inf}, a large constant. Then this loss component is

$$J_4^{(i)} = \frac{1}{B} \sum_{j=1}^{B} \left\| f\left(t_j^{(i)}, \bar{\boldsymbol{S}}_j{}^{(i)}, V_j^{(i)}\right) \right\|_2^2$$

4. $V \to 0$: [not done yet]

5. $V \to \infty$: with infinite volatility, it is almost sure that the intrinsic value of the option would reach the strike at some time before expiry, and thus the value of the option is the strike for the put. To implement this, set V_{inf} a large constant, and we have

$$J_6^{(i)} = \frac{1}{B} \sum_{j=1}^{B} \left\| f\left(t_j^{(i)}, \boldsymbol{S}_j{}^{(i)}, V_{\text{inf}}\right) - K \right\|_2^2$$

6.4.4 Numerical results

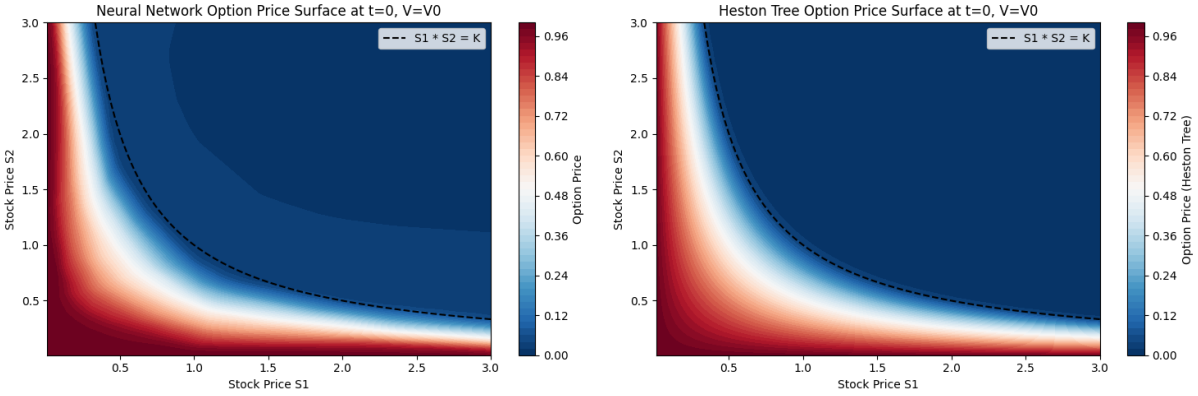

Figure 6.8 Neural network vs 1D reduction tree prices

Appendix A

Appendix 1

Bibliography

[1] F. Black and M. Scholes. The pricing of options and corporate liabilities. *Journal of political economy*, 81(3):637–654, 1973.

[2] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.

[3] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

[4] Z. Hu, K. Shukla, G. E. Karniadakis, and K. Kawaguchi. Tackling the curse of dimensionality with physics-informed neural networks. *Neural Networks*, 176:106369, 2024.

[5] A. Dhiman and Y. Hu. Physics informed neural network for option pricing. *arXiv preprint arXiv:2312.06711*, 2023.

[6] S. Wang, S. Sankaran, H. Wang, and P. Perdikaris. An expert's guide to training physics-informed neural networks. *arXiv preprint arXiv:2308.08468*, 2023.

[7] J. C. Cox, S. A. Ross, and M. Rubinstein. Option pricing: A simplified approach. *Journal of financial Economics*, 7(3):229–263, 1979.

[8] H. Zheng. Fundamentals of options pricing (lecture notes), 2025.

[9] Y. Zhao and H. Zheng. Fractional-boundary-regularized deep galerkin method for variational inequalities in mixed optimal stopping and control. *arXiv preprint arXiv:2505.19309*, 2025.

[10] S.-S. Jo et al. Variational inequality for perpetual american option price and convergence to the solution of the difference equation. *arXiv preprint arXiv:1903.05189*, 2019.

[11] P. Glasserman. *Monte Carlo methods in financial engineering*. Springer, 2004.

[12] R. A. Adams and J. J. Fournier. *Sobolev spaces*, volume 140. Elsevier, 2003.

[13] E. Di Nezza, G. Palatucci, and E. Valdinoci. Hitchhiker's guide to the fractional sobolev spaces. *Bulletin des sciences mathématiques*, 136(5):521–573, 2012.

[14] S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The review of financial studies*, 6(2):327–343, 1993.

[15] J. Gatheral. *The volatility surface: a practitioner's guide*. John Wiley & Sons, 2011.

[16] M. Vellekoop and H. Nieuwenhuis. A tree-based method to price american options in the heston model. *The Journal of Computational Finance*, 13(1):1–21, 2009.

[17] W. H. Press. *Numerical recipes 3rd edition: The art of scientific computing*. Cambridge university press, 2007.

[18] P. Bressloff. Introduction to stochastic differential equations and stochastic processes (lecture notes), 2025.

[19] H. Pham. *Continuous-time stochastic control and optimization with financial applications*, volume 61. Springer Science & Business Media, 2009.